

爱予慧AI语音交互服务系统 V1.0

设计说明书

著作权人：上海爱予慧科技有限责任公司

版本号：V1.0

文档版本：2026-09-01

一、引言

1.1 编写目的

本设计说明书旨在全面阐述"爱予慧AI语音交互服务系统 V1.0"（以下简称"本系统"）的总体架构、模块划分、数据流转、接口协议与关键技术设计，为软件著作权登记提供技术文档依据，并为后续的维护、演进与二次开发提供设计参照。本系统由上海爱予慧科技有限责任公司基于开源项目 xiaozhi-esp32-server（MIT 协议，著作权人 xinnan-tech）二次开发形成，在语音流水线编排、设备连接管理、Provider 工厂模式、记忆与知识图谱、内容安全门控等方面新增了大量独创代码，形成面向 AI 语音陪伴硬件的完整后端服务能力。

1.2 项目背景

随着 AI 语音陪伴硬件（ESP32 智能终端）的普及，需要一个稳定、可扩展的后端服务承载实时语音对话能力。本系统面向 ESP32 智能设备提供 WebSocket 实时音频通道与 HTTP 管理接口，完成从语音活动检测、语音识别、意图理解、大模型推理到语音合成的全链路语音 AI 交互，并对接智控台管理平台实现运行时参数动态下发。系统支持多服务商热切换、流式低延迟对话、长期记忆与用户画像、工具函数调用扩展等核心能力，是 AI 硬件产品的语音中枢。

本系统所在仓库为 xiaozhi-esp32-server，是 [xiaozhi-esp32](https://github.com/78/xiaozhi-esp32) 开源智能硬件项目的后端服务。上海爱予慧科技有限责任公司在取得上游 MIT 许可后，对其进行了大规模二次开发：新增了 AEC 回声消除机制、音频速率控制器、输出门控内容安全、统一工具处理器、PowerMem 记忆与知识图谱集成、三层配置远程热更新、声纹识别与多用户记忆隔离、GC 管理等独创模块，并扩展了 ASR/TTS/LLM 三大能力的 Provider 适配矩阵至数十家服务商。

1.3 术语定义

术语	说明
ASR	Automatic Speech Recognition，自动语音识别，将 PCM 音频流转写为文本
TTS	Text To Speech，语音合成，将文本转换为音频流
LLM	Large Language Model，大语言模型，负责对话推理与生成
VAD	Voice Activity Detection，语音活动检测，判断音频帧是否含人声
AEC	Acoustic Echo Cancellation，声学回声消除，剔除设备自身播放音对收音的干扰
Provider	提供者模式，以抽象基类定义能力、以工厂函数按配置实例化具体实现的解耦方式

Opus	低延迟语音编码格式，ESP32 设备与服务端音频传输的编码标准
OTA	Over The Air，空中固件升级，通过 HTTP 下发设备固件
MCP	Model Context Protocol，模型上下文协议，用于工具与模型间的能力对接
智控台	本系统配套的管理后台（manager-api + manager-web），通过 REST API 下发配置
PCM	脉冲编码调制，未压缩的原始音频采样数据
JWT	JSON Web Token，用于 HTTP 接口与 WebSocket 握手的紧凑型鉴权令牌
Dialogue	对话管理器，维护多轮对话历史与上下文截断
PowerMem	记忆 SDK，提供向量记忆存储与用户画像提取能力
GC	Garbage Collection，垃圾回收，本系统提供独立调度策略避免实时性停顿

1.4 参考标准

- 《计算机软件著作权登记办法》及中国版权保护中心 R11 登记指南
- GB/T 8567-2006 《计算机软件文档编制规范》
- MIT 开源许可证（上游 xiaozhi-esp32-server 许可证明）
- RFC 6455 WebSocket 协议
- HTTP/1.1（RFC 7230-7235）
- RFC 7519 JSON Web Token (JWT)

1.5 文档范围

本说明书覆盖本系统的总体架构、连接管理、语音交互流水线、消息处理、Provider 工厂、工具调用、HTTP 接口、记忆与图谱、配置系统、关键技术、部署运维与安全设计共十五章。源代码鉴别材料另册提交，与本说明书共同构成本系统完整的著作权登记文档。

二、系统概述

2.1 软件用途

本系统为 ESP32 智能硬件提供实时语音 AI 助手服务端能力。设备通过 WebSocket 建立长连接后，持续上传 Opus 编码音频流；系统完成语音活动检测、语音识别、意图理解、大模型推理与语音合成，将合成音频流式回传设备播放，实现自然流畅的端到端语音对话。同时系统通过 HTTP 端点

提供 OTA 固件升级与视觉分析能力。系统是 AI 语音陪伴硬件产品的后端中枢，承载对话、记忆、工具调用与设备管理四大核心业务。

2.2 运行环境

2.2.1 服务端环境

- 操作系统：Linux（Ubuntu 22.04 及以上）或 macOS（开发期）
- 运行时：Python 3.12
- 依赖服务：OceanBase（MySQL 兼容，容器名 seekdb，用于记忆向量与图谱存储）、Redis（缓存）
- 外部依赖：ffmpeg（Opus 编解码，启动时由 check_ffmpeg_installed 校验）
- 可选：GPU（本地 ASR/TTS/LLM 推理加速）、CUDA（torch 2.2.2）
- 网络：对外开放 WebSocket 端口（默认 8000）与 HTTP 端口（默认 8003）
- 容器：提供 Dockerfile 与 docker-compose-oceanbase.yml 一键集成部署

2.2.2 终端环境

- ESP32 系列智能终端设备，运行 xiaozhi-esp32 固件
- 支持 Opus 编解码、WebSocket 长连接与 OTA 升级
- 支持麦克风阵列与扬声器输出
- 支持可选 MQTT 网关接入（mqtt-gateway 转发）

2.3 技术栈

层次	技术选型	用途
异步框架	asyncio	双服务并发、连接 IO 与流水线编排
WebSocket	websockets	设备实时音频通道
HTTP	aiohttp	OTA 与视觉分析 HTTP 服务
ML 推理	torch 2.2.2 / funasr 1.2.7 / sherpa_onnx 1.12.29 / silero_vad 6.1.0	本地 ASR、VAD 模型
日志	loguru	结构化日志，配置于 config/logger.py

数据存储	pyobvector (OceanBase)	记忆向量与知识图谱
配置	YAML + 远程 REST API	三层配置递归合并
认证	PyJWT	HTTP 接口与 WebSocket 握手鉴权
音频编解码	ffmpeg (外部进程)	Opus 与 PCM 互转
记忆 SDK	PowerMem 1.1.0	长期记忆与用户画像

2.4 主要功能

1. 实时语音对话：WebSocket 承载双向音频流，完成 VAD→ASR→意图→LLM→TTS 全链路交互，流式处理首字延迟秒级。
2. 多 Provider 适配：ASR 14 种、TTS 20 种、LLM 11 种、VAD、意图、记忆 6 种能力均以工厂模式按配置实例化，支持数十家服务商热切换。
3. 工具与函数调用：UnifiedToolHandler 统一调度 IoT 控制、MCP 工具、服务端插件、服务端 MCP 工具四类来源，15 个内置函数插件自动发现注册。
4. OTA 固件升级：HTTP 端点 /xiaozi/ota/ 下发设备固件元数据与二进制包，支持 MD5 校验与 Token 鉴权。
5. 视觉分析：HTTP 端点 /mcp/vision/explain 接收图像，调用多模态 LLM 完成理解与描述。
6. 记忆与画像：基于 PowerMem 与 OceanBase 向量库/图谱存储用户长期记忆与画像特征，跨会话个性化。
7. 内容安全：输出门控机制对 LLM 输出在送入 TTS 前进行安全校验与阻断。
8. 远程配置：对接智控台 API，运行时拉取配置覆盖本地，无需重启即生效。
9. AEC 全双工：声学回声消除使设备播放语音时仍可被用户打断，支持自然插话。
10. 声纹识别：说话人识别联动记忆，支持同设备多用户独立记忆与画像。

2.5 用户角色

角色	说明
终端用户	通过 ESP32 设备与本系统语音对话的自然人
设备	运行 xiaozi-esp32 固件的 ESP32 终端，本系统的直接对接方
管理员	通过智控台配置智能体、模型参数、设备注册的运营人员

开发者	基于本系统二次开发或扩展 Provider/插件的技术人员
-----	-------------------------------

三、系统架构设计

3.1 总体架构

本系统采用双服务异步架构，由一个入口模块 app.py 同时启动 WebSocket 服务与 HTTP 服务，两者共享同一份配置与 Provider 实例池。WebSocket 服务面向设备实时音频交互，HTTP 服务面向管理与升级类无状态请求。双服务运行于同一 asyncio 事件循环，通过共享的 ConnectionHandler 与 Provider 工厂协同工作。该设计避免了跨进程通信开销，同时以协程隔离保证了各连接与各请求的独立性。

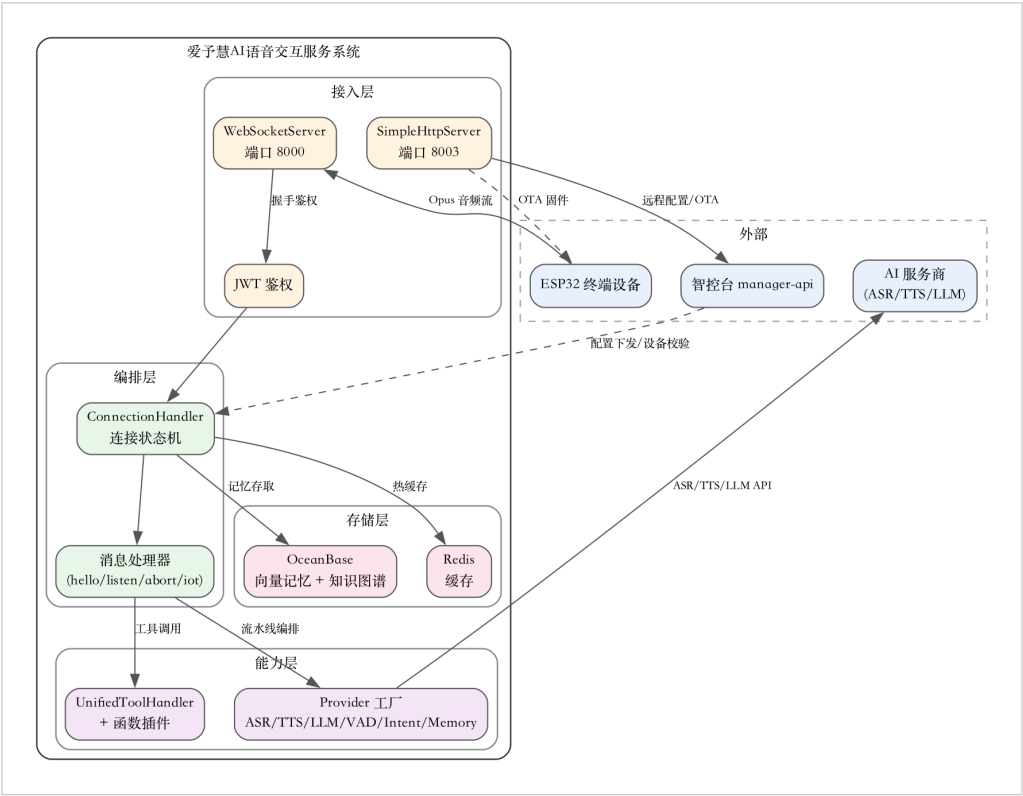


图 1 系统总体架构图（四层分层 + 外部依赖）

3.2 分层设计

系统自上而下分为接入层、编排层、能力层、存储层四层，层间通过抽象接口解耦，逐层职责单一。接入层只管协议与鉴权，编排层只管状态与流转，能力层只管具体 AI 算法的执行，存储层只管持久化与召回。层间依赖单向向下，上层不感知下层具体实现，通过工厂与基类间接调用。

层次	模块	职责
----	----	----

接入层	websocket_server / http_server / auth	协议接入、JWT 鉴权、连接生命周期
编排层	connection / handle	连接状态机、消息分发、流水线编排
能力层	core/providers/* （asr/tts/llm/vad/intent/memory/tools）	六大能力 Provider 实现与工具调度
存储层	OceanBase / Redis / 智控台 API	向量记忆、图谱、缓存、远程配置

3.3 Provider 模式

系统的核心设计是 Provider 模式。每种 AI 能力定义一个抽象基类（如 ASRProviderBase），声明统一接口；具体实现（如 AliyunASR、FunLocalASR、TencentASR 等）继承基类；由 core/utils/ 下的工厂函数（如 asr.create_instance(class_name)）根据配置键动态实例化。该设计使得服务商切换仅需修改配置而无需改动业务代码，符合开闭原则与依赖倒置原则。

能力	抽象基类	工厂函数	配置键	实现数
ASR	ASRProviderBase	core/utils/asr.py: create_instance	selected_module.ASR	14
TTS	TTSPProviderBase	core/utils/tts.py: create_instance	selected_module.TTS	20
LLM	LLMProviderBase	core/utils/llm.py: create_instance	selected_module.LLM	11
VAD	VADProviderBase	core/utils/vad.py: create_instance	selected_module.VAD	2
Intent	IntentProviderBase	core/utils/intent.py: create_instance	selected_module.Intent	2
Memory	MemoryProviderBase	core/utils/memory.py: create_instance	selected_module.Memory	6

Provider 模块在 core/utils/modules_initialize.py 中按需懒加载初始化，避免启动时全量初始化带来的内存与时间开销。已初始化的实例按 client_id 缓存，同设备重连可复用。

3.4 目录结构

```
main/xiaozhi-server/
├─ app.py                # 入口，启动双服务
├─ config.yaml           # 基础配置（已提交）
├─ config/
│   ├─ config_loader.py  # 三层配置递归合并
│   ├─ settings.py       # load_config 入口
│   └─ logger.py         # loguru 日志配置
├─ models/               # 本地模型文件（SenseVoiceSmall 等）
├─ core/
│   ├─ connection.py     # ConnectionHandler 连接状态机
│   ├─ websocket_server.py
│   ├─ http_server.py
│   └─ auth.py           # JWT 认证
```

```

├── api/                # OTA / 视觉分析 HTTP 处理器
│   ├── base_handler.py
│   ├── ota_handler.py
│   └── vision_handler.py
├── handle/             # 消息处理器
│   ├── receiveAudioHandle.py
│   ├── sendAudioHandle.py
│   ├── intentHandler.py
│   ├── helloHandle.py / abortHandle.py
│   ├── reportHandle.py / textHandle.py
│   └── textMessageHandlerRegistry.py
├── providers/          # 六大能力 Provider 实现
│   ├── asr/ (14 实现 + base + dto + utils)
│   ├── tts/ (20 实现 + base + dto)
│   ├── llm/ (11 实现 + base + system_prompt)
│   ├── vad/ intent/ memory/ (6 实现)
│   ├── tools/ (7 工具源 + unified_tool_handler)
│   ├── content_safety/
│   └── vllm/
├── utils/              # 工厂、对话、工具函数
│   ├── asr.py tts.py llm.py vad.py intent.py memory.py
│   ├── dialogue.py audioRateController.py opus_encoder_utils.py
│   ├── content_safety.py gc_manager.py vllm.py
│   ├── voiceprint_provider.py wakeup_word.py prompt_manager.py
│   ├── modules_initialize.py util.py auth.py
│   └── output_counter.py textUtils.py p3.py
├── plugins_func/       # 自动发现的函数插件
│   ├── register.py     # @register_function 注册表
│   ├── loadplugins.py  # auto_import_modules 扫描
│   └── functions/      # 15 个内置能力

```

3.5 模块依赖关系

app.py 依赖 config.settings.load_config 与 config.logger.setup_logging 完成初始化，随后实例化 WebSocketServer 与 SimpleHttpServer。WebSocketServer 依赖 ConnectionHandler 处理每条连接；ConnectionHandler 依赖 core/handle 下的处理器与 core/providers 下的 Provider，并通过 core/utils 工厂实例化。core/api 下的 HTTP 处理器依赖 core/auth 完成鉴权。整体依赖单向，无循环引用。

四、连接管理模块设计

4.1 ConnectionHandler 状态机

ConnectionHandler (core/connection.py, 约 2000 行) 是本系统最核心的模块，每个 WebSocket 连接对应一个独立实例，承载该连接全生命周期的状态与行为。它是一个典型的状态机，管理音频

I/O 队列、TTS 合成队列、对话历史、认证态、设备绑定态、记忆上下文与工具调用上下文。该模块是整个语音交互流水线的编排枢纽，所有 Provider 实例、队列、协程均挂载于其上。

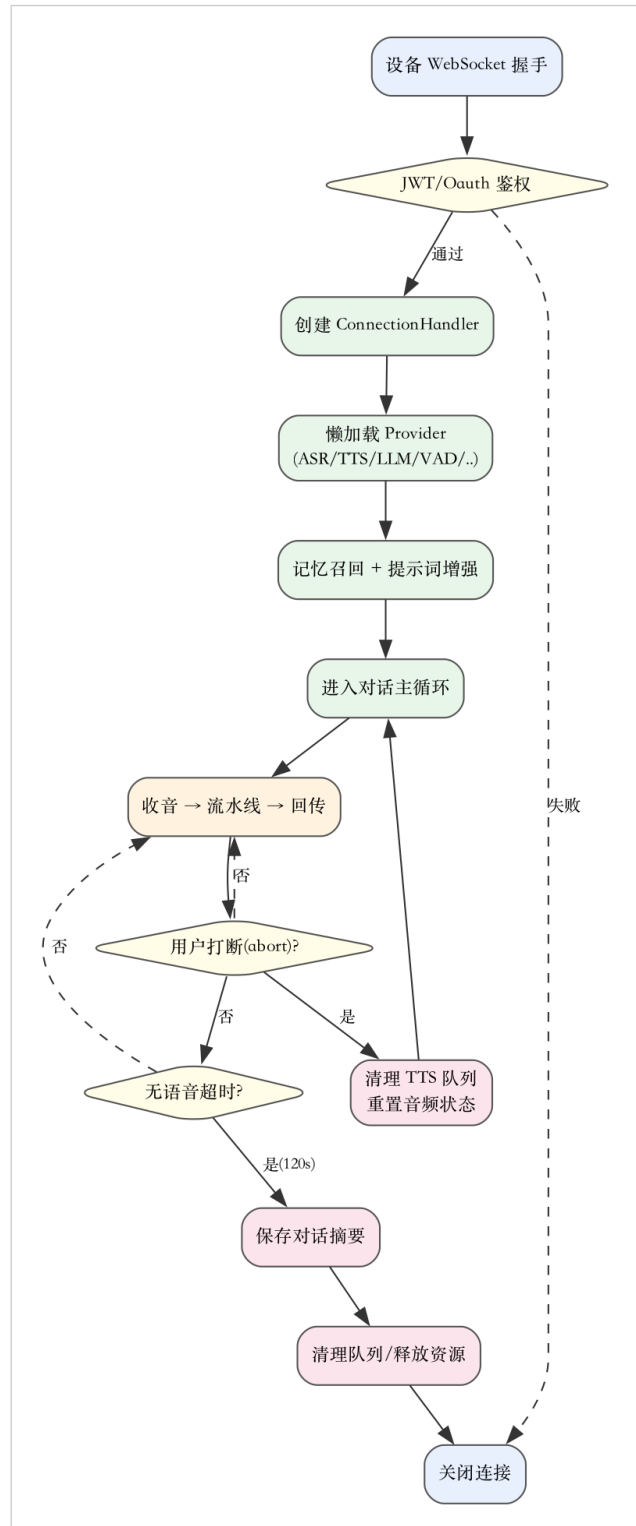


图 2 连接生命周期流程图（握手→初始化→对话→超时→关闭）

4.1.1 核心状态

状态字段	类型	说明
asr_audio_queue	asyncio.Queue	待识别的 PCM 音频帧队列
tts_audio_queue	Queue	待播放的 Opus 音频包队列，含句子类型标记
dialogue	Dialogue	对话历史管理器，维护多轮上下文
client_id	str	设备唯一标识
client_secret	str	设备密钥，鉴权用
connected	bool	连接是否存活
asr / tts / llm / vad / intent / memory	Provider	该连接绑定的能力实例
tool_handler	UnifiedToolHandler	统一工具调用处理器
vad_resume_task	asyncio.Task	VAD 恢复检测协程
speak_status	SpeakStatus	播放进度跟踪

4.1.2 生命周期方法

```
class ConnectionHandler:
    def __init__(self, config, ws, client_id, client_secret): ...

    async def handle_connection(self, ws):
        # 主循环: 接收消息 → _route_message 路由 → 处理 → 回送

    async def _route_message(self, message):
        # 文本 → TextMessageHandlerRegistry 分发
        # 二进制 → _process_websocket_audio

    def _initialize_components(self):
        # 懒加载 ASR/TTS/LLM/VAD/Intent/Memory Provider

    async def _background_initialize(self):
        # 后台异步初始化，避免阻塞首帧

    def _initialize_memory(self):      # 记忆 Provider 初始化与召回
    def _initialize_intent(self):     # 意图 Provider 初始化
    def _initialize_asr(self):        # ASR Provider 初始化
    def _initialize_tts(self):        # TTS Provider 初始化
    def _initialize_voiceprint(self): # 声纹 Provider 初始化
    def _init_prompt_enhancement(self): # 提示词动态增强
    def _inject_tool_call_fewshot(self): # 工具调用示例注入
```

```
async def chat(self, query, depth=0, safety_context=None, output_gate=None):
    # 对话主入口：意图→LLM→工具→TTS 编排

def _handle_function_result(self, ...): # 工具结果回灌
async def close(self, ws=None):        # 清理退出
def clear_queues(self):                 # 队列清理
def reset_audio_states(self):           # 音频状态重置
```

4.2 认证与设备绑定

WebSocket 握手阶段由 `WebSocketServer._handle_auth` 完成。设备在握手时携带 JWT 或 OAuth-Token，服务端校验 `auth_key` 签名或调用智控台 API 验证设备合法性。认证通过后分配 `client_id` 并创建 `ConnectionHandler`。未绑定设备进入待绑定提示流程，由 `_discard_message_with_bind_prompt` 引导用户完成绑定。`auth_key` 同时用于 OTA Token 生成与视觉接口 JWT 签发，是系统根密钥。

`auth_key` 优先级：配置 `server.auth_key` > 智控台 `manager-api.secret` > 随机 `uuid4.hex`。随机密钥仅用于无智控台的本地开发，生产环境必须显式配置以保证多实例一致。

4.3 音频 I/O 与 Opus 编解码

设备上传的音频为 Opus 编码包，服务端在 `_decode_opus_packet` 中调用 `ffmpeg` 解码为 PCM 帧。回传音频同样经 Opus 编码后发送。Opus 编解码隔离在 `core/utils/opus_encoder_utils.py`，保证业务层只处理 PCM。编码参数（采样率 16kHz、声道数、比特率）可配置，适配不同网络条件。编解码通过子进程调用 `ffmpeg`，避免阻塞事件循环。

4.4 AEC 回声消除

设备扬声器播放的合成语音会被麦克风重新采集，若不消除将导致 ASR 把系统输出识别为用户输入，形成“自言自语”。系统在 `_apply_aec` 中缓存近期播放的 PCM 帧，与当前收音帧做对齐相减，剔除回声成分。`_check_aec_cache_expiry` 定时清理过期缓存，防止内存膨胀。该机制使设备在播放语音的同时仍可被唤醒打断，实现全双工对话。AEC 缓存按时间戳对齐，保证消除精度。

4.5 MQTT 音频桥接

除 WebSocket 外，系统支持通过 MQTT 通道接收音频消息，由 `_process_mqtt_audio_message` 处理。该路径用于非 WebSocket 组网场景（如经 `mqtt-gateway` 转发的局域网设备），消息统一汇入 `_put_asr_audio_with_aec` 与 `_put_asr_audio` 进入 ASR 流水线，保持下游处理一致。该设计使系统能同时服务直连与网关转发两类设备。

4.6 超时与无语音关闭

系统对空闲连接设超时保护：close_connection_no_voice_time（默认 120 秒）无语音输入即关闭连接，释放资源。_check_timeout 协程定时检测，超时触发 _save_and_close。tts_timeout（默认 10 秒）保护 TTS 合成不至无限等待，tool_call_timeout（默认 30 粘）保护工具调用不至阻塞对话。三类超时共同保障系统在异常客户端下的稳定性。

4.7 重启与重连

handle_restart 处理设备主动重启请求，清理当前连接状态后重新初始化。_init_connection_state 重置连接级状态，_save_and_close 保存对话摘要后关闭。设备重连时按 client_id 复用已初始化的 Provider 实例，避免重复加载模型的昂贵开销。

五、语音交互流水线设计

5.1 流水线总览

语音交互是本系统的核心数据流，由 core/handle/ 下的处理器协同完成。一帧音频从设备到达，到合成语音回传，依次经过 VAD、ASR、意图、LLM、TTS 五个阶段，每个阶段均为流式处理，首字延迟可控制在秒级。流水线各阶段通过 asyncio 队列衔接，前一阶段产出即可被后一阶段消费，形成流水线并发，而非串行等待。

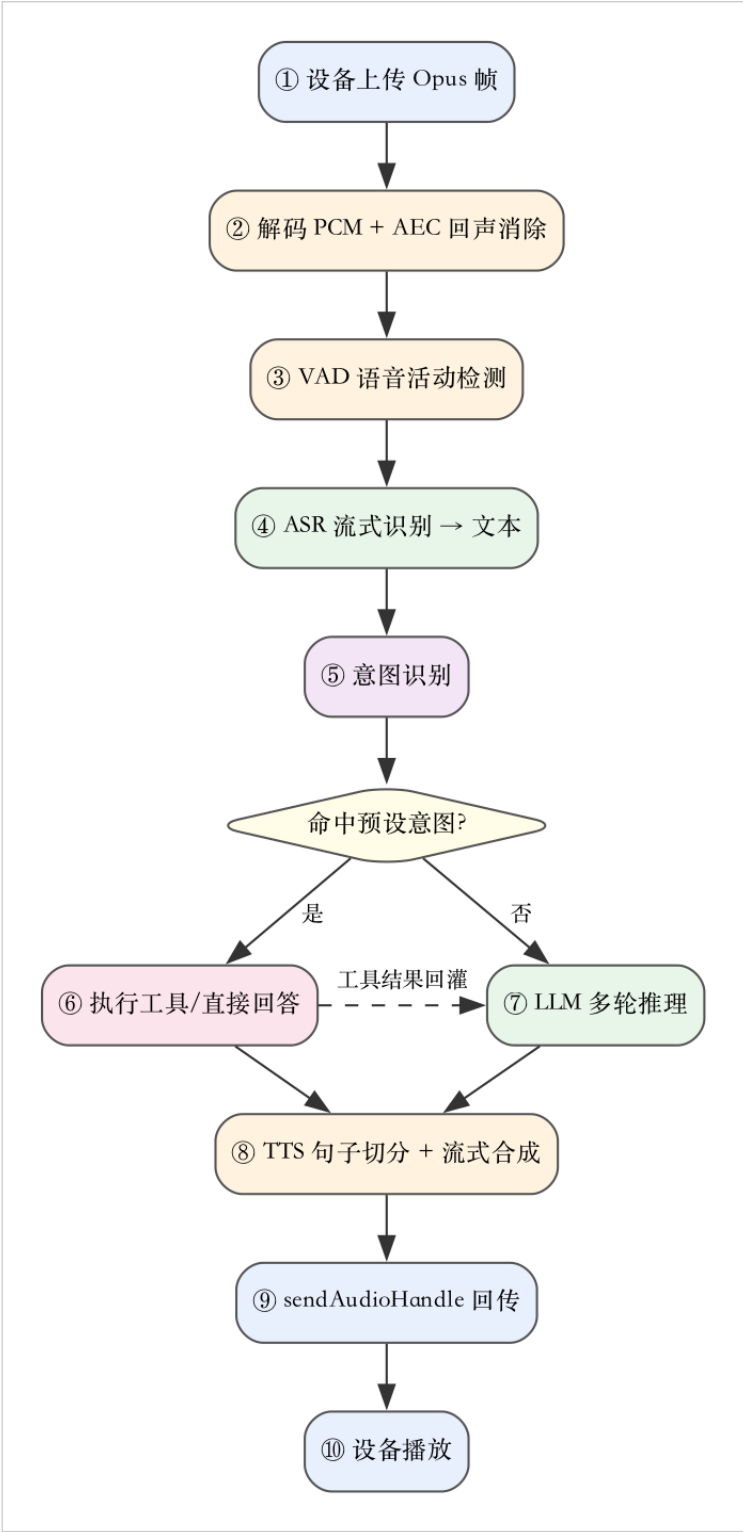


图 3 语音交互时序图（VAD→ASR→意图→LLM→TTS 全链路）

5.2 VAD 语音活动检测

VAD (core/providers/vad) 判断 PCM 帧是否含人声。当连续帧判定为语音时触发 ASR 开始识别；判定静音时触发 ASR 结束并产出文本。receiveAudioHandle 调用 conn.vad.is_vad(conn, pcm_frame) 完成逐帧判断。当 VAD 暂停（如设备播放中）时，resume_vad_detection 协程在播放结

束后恢复检测。VAD 默认基于 silero_vad 模型，亦支持基于能量的 SimpleVAD 实现。VAD 的灵敏度与静音判定阈值可配置。

5.3 ASR 语音识别

ASR Provider 接收 PCM 帧流式识别，支持本地 FunASR/SenseVoice/sherpa_onnx 与云端阿里云/腾讯云/百度/讯飞/豆包/OpenAI/Vosk 等多种实现。conn.asr.receive_audio(conn, pcm_frame, have_voice) 持续喂入音频，识别结果通过回调汇入意图处理。VAD 与 ASR 解耦：VAD 只管切分语音段，ASR 只管把段转写为文本。流式 ASR 在用户说话过程中即开始产出部分结果，配合 VAD 静音判定确认最终文本。

ASR Provider 实现清单 (core/providers/asr/) : aliyun_stream、aliyun、aliyunbl_stream、baidu、doubao_stream、doubao、fun_local、fun_server、openai、qwen3_asr_flash、sherpa_onnx_local、tencent、vosk、xunfei_stream，共 14 种实现，覆盖本地推理与主流云服务商。

5.4 意图识别

intentHandler.handle_user_intent 在 ASR 产出文本后先行判断：若命中预设意图（如控制设备、查询天气、退出对话），直接执行对应工具并产出响应，跳过 LLM 以降低延迟与成本；若未命中，文本进入 LLM 对话。意图识别由 core/providers/intent 的 Provider 实现，支持基于关键词的 KeywordIntent 与基于大模型的 LLMIntent 两种策略。意图命中后可调用 speak_trusted_text 直接产出可信文本，不经 LLM 生成，保证特定指令的即时响应。

5.5 LLM 大模型推理

LLM Provider (core/providers/llm) 接收对话历史与系统提示词，流式产出回复文本。系统在 ConnectionHandler.chat 中编排完整对话：构造 messages → 调用 LLM → 流式接收 → 实时送入 TTS → 处理工具调用。_handle_function_result 负责把工具执行结果回灌 LLM 继续生成。LLM 支持 OpenAI 兼容接口、本地 vLLM、通义千问、DeepSeek、智谱、Dify、FastGPT、Gemini、Ollama、Xinference、Coze、HomeAssistant 等共 11 类实现。system_prompt.py 模块支持运行时动态系统提示词管理。

5.6 TTS 语音合成与句子切分

TTS Provider (core/providers/tts) 将 LLM 流式产出的文本按标点切分为句子，边生成边合成，产出的 Opus 包按 FIRST / MIDDLE / LAST 三态标记入队 tts_audio_queue。该设计使设备能在首句合成完成即开始播放，无需等待整段文本合成完毕。sendAudioHandle 按句子序号有序消费队列回传，保证播放顺序正确。TTS 实现清单 (core/providers/tts/) 共 20 种：alibl_stream、

aliyun_stream、aliyun、cozecn、custom、default、doubao、edge、fishspeech、gpt_sovits_v2、gpt_sovits_v3、huoshan_double_stream、index_stream、minimax_httpstream、openai、paddle_speech、siliconflow、tencent、xunfei_stream，支持流式合成与音色克隆。

5.7 流式与并发控制

流水线各阶段通过 `asyncio` 协程与队列衔接，ASR、LLM、TTS 并发执行而非串行等待。`audioRateController` (`core/utils/audioRateController.py`) 控制回传音频的速率，避免设备缓冲溢出；`output_counter` 统一管理输出序号，支持内容安全门控按序阻断或释放。当 LLM 流式产出与 TTS 合成速度不匹配时，速率控制器动态调节发送间隔，保证平滑播放。

5.8 中断与 abort

用户可在对话进行中打断 (abort)。`abortHandle` 接收打断指令后，清理 TTS 队列 (`clearSpeakStatus`)、重置音频状态 (`reset_audio_states`)、停止当前 LLM 生成。AEC 机制保证打断时设备收音不受自身播放影响。中断后流水线立即就绪，接受下一轮对话，实现自然插话体验。

六、消息处理设计

6.1 WebSocket 文本协议

设备与服务端通过 `WebSocket` 交换 `JSON` 文本消息与二进制音频帧。文本消息由 `TextMessageHandlerRegistry` (`core/handle/textMessageHandlerRegistry.py`) 按 `type` 字段分发到对应处理器。消息类型由 `TextMessageType` 枚举约束，非法类型拒绝处理。

消息类型	枚举	处理器	用途
hello	<code>TextMessageType.HELLO</code>	<code>helloHandle</code>	握手问候，建立会话，交换设备能力
listen	<code>TextMessageType.LISTEN</code>	<code>textMessageHandler</code>	监听控制（开始/停止收音）
abort	<code>TextMessageType.ABORT</code>	<code>abortHandle</code>	中断当前对话与合成
iot	<code>TextMessageType.IOT</code>	<code>textMessageHandler</code>	IoT 设备状态上报与控制
mcp	<code>TextMessageType.MCP</code>	<code>textMessageHandler</code>	MCP 工具调用
server	<code>TextMessageType.SERVER</code>	<code>textMessageHandler</code>	服务端控制指令
ping	<code>TextMessageType.PING</code>	<code>textMessageHandler</code>	心跳保活

6.2 文本消息处理器

TextMessageProcessor 负责文本消息的预处理与统一调度，textMessageType 定义消息类型枚举，textHandler 子目录存放各类型的具体处理逻辑。处理器可访问 ConnectionHandler 的全部状态，包括对话历史、Provider 实例与设备信息。textMessageHandler 作为通用分发器，根据消息体内容进一步路由到具体子处理逻辑。

6.3 二进制音频处理

二进制帧由 _process_websocket_audio 接收，解码为 PCM 后入 ASR 队列。音频帧携带时间戳，用于 AEC 对齐与速率控制。发送侧由 sendAudioHandle 从 tts_audio_queue 取包，按序号回传设备，并维护 SpeakStatus 跟踪播放进度。_put_asr_audio 与 _put_asr_audio_with_aec 是音频入队的两条路径，后者在入队前先做回声消除。

6.4 报告与上报

reportHandle 处理设备上报的状态信息（如音量、电量、网络），_report_worker 协程在后台消费上报队列，_process_report 将报告写入日志与智控台。该通道也用于对话摘要的持久化，由 _log_dialogue_summary 在每轮对话结束后生成摘要写入记忆。上报通道与对话通道隔离，互不阻塞。

6.5 消息序列示例

一次完整语音对话的消息序列如下：

1. 设备 → 服务端: {"type": "hello", ...} 握手
2. 服务端 → 设备: {"type": "hello", ...} 确认
3. 设备 → 服务端: 二进制 Opus 帧 ×N 收音
4. 服务端 → 设备: {"type": "listen", state: "start"} 开始识别
5. 服务端 → 设备: 二进制 Opus 帧 ×M 回传合成音频
6. 服务端 → 设备: {"type": "listen", state: "stop"} 识别结束
7. 设备 → 服务端: {"type": "ping"} 心跳

七、Provider 工厂设计

7.1 工厂模式实现

所有 Provider 通过工厂函数实例化。工厂函数接收配置中的类名，动态导入对应模块并构造实例。以 ASR 为例：


```
# core/utils/asr.py
def create_instance(class_name: str, *args, **kwargs) -> ASRProviderBase:
    module = importlib.import_module(
        f"core.providers.asr.{class_name.lower()}"
    )
    cls = getattr(module, class_name)
    return cls(*args, **kwargs)
```

该模式使新增服务商只需在 `core/providers/<能力>/` 下新增实现类并配置类名，无需改动工厂与业务代码，符合开闭原则。工厂函数统一返回抽象基类类型，业务层面向接口编程而非具体实现。

7.2 懒加载与后台初始化

Provider 不在启动时全部构造，而在连接首次需要时由 `_initialize_components` 按需创建。重型 Provider（如本地 ASR 模型加载）通过 `_background_initialize` 在后台协程中初始化，避免阻塞首个音频帧的处理。已初始化的实例按 `client_id` 缓存，同设备重连可复用。`modules_initialize.py` 是统一初始化入口，按依赖顺序初始化各 Provider。

7.3 ASR Provider 实现清单

ASR (`core/providers/asr/`) 共 14 种实现，按部署形态分两类：

类型	实现	说明
本地推理	<code>fun_local</code> 、 <code>fun_server</code> 、 <code>sherpa_onnx_local</code> 、 <code>vosk</code>	基于 <code>funasr/SenseVoice/sherpa_onnx</code> 本地模型
云端流式	<code>aliyun_stream</code> 、 <code>aliyun</code> 、 <code>aliyunbl_stream</code> 、 <code>baidu</code> 、 <code>doubao_stream</code> 、 <code>doubao</code> 、 <code>openai</code> 、 <code>qwen3_asr_flash</code> 、 <code>tencent</code> 、 <code>xunfei_stream</code>	对接阿里云/百度/豆包/OpenAI/通义/腾讯/讯飞云服务

7.4 TTS Provider 实现清单

TTS (`core/providers/tts/`) 共 20 种实现，覆盖本地与云端、流式与非流式、音色克隆：

类型	实现	说明
本地合成	<code>default</code> 、 <code>paddle_speech</code> 、 <code>gpt_sovits_v2</code> 、 <code>gpt_sovits_v3</code> 、 <code>fishspeech</code>	本地推理合成，含音色克隆
云端流式	<code>aliyun_stream</code> 、 <code>alibl_stream</code> 、 <code>huoshan_double_stream</code> 、 <code>index_stream</code> 、 <code>minimax_httpstream</code> 、 <code>xunfei_stream</code> 、 <code>cozecn</code> 、 <code>siliconflow</code>	流式合成，首句低延迟

云端非流式	aliyun、doubao、tencent、openai、edge、custom	整段合成后回传
-------	--	---------

7.5 LLM Provider 实现清单

LLM (core/providers/llm/) 共 11 类实现，含本地推理与多云对接：

实现	说明
openai	OpenAI 兼容接口（通义、DeepSeek、智谱、Moonshot 等）
ollama	本地 Ollama 推理
xinference	本地 Xinference 推理
gemini	Google Gemini
dify	Dify 平台对接
fastgpt	FastGPT 平台对接
coze	扣子平台对接
homeassistant	Home Assistant 对话集成
AliBL	阿里百炼

另含 system_prompt.py 提供系统提示词的动态管理与增强。

7.6 Memory Provider 实现清单

Memory (core/providers/memory/) 共 6 种实现：

实现	说明
powermem	基于 PowerMem SDK，OceanBase 向量与图谱存储，支持用户画像
volmem0	基于 mem0ai 的变体
mem0ai	mem0ai SDK 记忆
mem_local_short	本地短期记忆
mem_report_only	仅对话摘要持久化
nomem	无记忆模式（开发调试用）

7.7 VAD 与 Intent Provider

VAD 共 2 种：SileroVAD（基于 silero_vad 模型）、SimpleVAD（基于能量阈值）。Intent 共 2 种：LLMIntent（基于大模型判断）、KeywordIntent（基于关键词匹配）。两类能力实现精简但接口完整，支持按场景选用。

八、工具与函数调用设计

8.1 统一工具处理器

UnifiedToolHandler（core/providers/tools/unified_tool_handler.py）是工具调用的统一调度入口，协调四类工具来源：IoT 设备控制、MCP 工具、服务端插件、服务端 MCP 工具。UnifiedToolManager（unified_tool_manager.py）负责工具的注册、发现与生命周期管理。当 LLM 决定调用工具时，工具名与参数经此处理器分发到对应执行器，执行结果回灌 LLM 继续生成。

8.2 工具源清单

工具源	目录	说明
base	core/providers/tools/base	工具基类与抽象
device_iot	device_iot	IoT 设备控制工具
device_mcp	device_mcp	设备侧 MCP 工具
mcp_endpoint	mcp_endpoint	MCP 端点对接
server_mcp	server_mcp	服务端 MCP 工具
server_plugins	server_plugins	服务端插件工具

8.3 函数插件自动发现

plugins_func/实现自动发现式函数注册表。
loadplugins.auto_import_modules("plugins_func.functions") 在启动时扫描 plugins_func/functions/ 下所有模块并导入，模块内通过装饰器注册能力：

```
# 注册一个函数能力
@register_function(name="play_music", desc="播放音乐",
                  type=ToolType.SYS_PLAYER)
class PlayMusicFunction(BaseFunction):
    async def execute(self, **kwargs): ...
```

该设计使新增工具能力只需新增一个模块文件，无需修改注册逻辑或工具调度器，极大降低了扩展成本。系统启动后所有 @register_function 装饰的类自动纳入工具表，对 LLM 透明可用。

8.4 内置函数插件清单

plugins_func/functions/ 共 15 个内置插件：

插件	能力
play_music	音乐播放
get_weather	天气查询（需 api_key）
get_news_from_chinanews	社会/科技/财经新闻
get_news_from_newsnow	多类型新闻聚合
get_time	时间查询
handle_exit_intent	退出意图处理
change_role	角色切换
call_device	设备呼叫
web_search	联网搜索
search_from_ragflow	RAGFlow 知识库检索
hass_init / hass_get_state / hass_set_state / hass_play_music	Home Assistant 集成

8.5 MCP 协议支持

系统支持以 MCP（Model Context Protocol）对接外部工具集。MCP 端点在配置中声明，core/ utils/util.validate_mcp_endpoint 校验其合法性。MCP 工具与本地插件统一纳入 UnifiedToolHandler，对 LLM 透明，LLM 无需感知工具来源即可调用。该设计使系统能动态接入第三方工具生态而不改动核心代码。

8.6 工具调用与对话集成

在 ConnectionHandler.chat 中，LLM 流式输出期间若触发工具调用，_handle_function_result 暂停文本送入 TTS，执行工具后将结果构造为新的 tool 消息回灌 LLM，LLM 基于结果继续生成自然语言回复。多工具调用通过 _merge_tool_calls 合并去重，保证幂等。_extract_direct_answer_response 与 _clean_response_garbage 对工具直接回答与模型输出垃圾文本做清洗，保证回传质量。

九、HTTP 接口设计

9.1 HTTP 服务

SimpleHttpServer (core/http_server.py) 基于 aiohttp 构建，监听默认端口 8003，提供 OTA 与视觉分析两类端点。路由在启动时通过 app.add_routes 注册，base_handler.py 提供通用请求处理基类。服务与 WebSocket 共享事件循环与配置。

9.2 OTA 固件升级

OtaHandler (core/api/ota_handler.py) 提供 /xiaozihi/ota/ 端点。设备请求时携带当前版本与设备型号，服务端返回最新固件元数据（版本号、大小、MD5、下载地址）；设备按元数据请求 .bin 文件完成下载升级。OTA Token 通过 auth_key 签名生成，保证升级请求合法。固件包带 MD5 校验，设备下载后验证完整性。

```
GET /xiaozihi/ota/?device_id=xxx&version=1.0.0
Response:
{
  "version": "1.1.0",
  "size": 1234567,
  "md5": "a1b2c3...",
  "url": "https://.../firmware-1.1.0.bin"
}
```

OTA 端点同时用于下发 WebSocket 地址与 MQTT 网关配置，使设备在出厂后通过 OTA 完成服务地址绑定，无需预置。

9.3 视觉分析

VisionHandler (core/api/vision_handler.py) 提供 /mcp/vision/explain 端点，接收图片（Base64 或 URL），调用多模态 LLM 生成图像描述。请求通过 JWT 鉴权，Token 由 auth_key 签发。该能力作为 MCP 工具供设备在对话中调用，实现“看图说话”，扩展了语音交互的多模态能力。

9.4 鉴权设计

HTTP 接口采用 JWT 鉴权。系统启动时确定 auth_key，优先级为：配置文件 server.auth_key > 智控台 manager-api.secret > 随机生成 uuid4.hex。auth_key 同时用于 WebSocket 握手鉴权、OTA Token 生成与视觉接口 JWT 签发，是系统的根密钥。鉴权逻辑隔离在 core/auth.py 与 core/utils/auth.py。生产环境必须显式配置 auth_key，避免随机密钥导致多实例不一致。

9.5 接口规范

端点	方法	鉴权	用途
/xiaozhi/ota/	GET	OTA Token	固件元数据查询
/xiaozhi/ota/	GET	OTA Token	固件二进制下载
/mcp/vision/explain	POST	JWT	视觉分析

十、记忆与知识图谱设计

10.1 记忆模块

Memory Provider (core/providers/memory) 基于 PowerMem SDK 实现长期记忆。每轮对话结束后，_log_dialogue_summary 生成对话摘要，提取值得记忆的事实（用户偏好、事实、事件），向量化后存入 OceanBase 向量库。下一轮对话前，_initialize_memory 根据当前输入召回相关记忆，注入 LLM 上下文，实现跨会话的个性化。记忆是语音陪伴产品"记得用户"体验的技术基础。

10.2 用户画像

UserMemory 自动从对话中提取用户画像特征（性格、兴趣、关系网络等），结构化存入知识图谱。画像随对话持续更新，使系统能够根据用户特征调整对话风格与内容推荐。画像提取基于 PowerMem 的 user_profile_prompts 提示词模板，由 LLM 完成结构化抽取。

10.3 知识图谱存储

知识图谱基于 OceanBase 图存储 (pyobvector)。节点为实体（人、物、概念），边为关系。图谱与向量库协同：向量库负责语义相似召回，图谱负责关系推理。两者共同构成设备的长期记忆中枢。oceanbase_graph.py 管理图谱的读写，oceanbase_vector.py 管理向量的读写。

10.4 记忆与对话集成

记忆召回发生在 chat 方法入口：输入文本 → 记忆 Provider 召回相关片段 → 拼入 system prompt 或 messages → 送入 LLM。prompt_manager 与 _init_prompt_enhancement 将召回的记忆与用户画像增强进系统提示词，使 LLM 在生成时具备用户历史上下文。该机制使设备"记住"用户跨次对话的关键信息。

10.5 记忆 Provider 选型

生产环境默认 powermem (PowerMem + OceanBase)，开发环境可用 mem_local_short (本地短期) 或 nomem (无记忆)。mem_report_only 仅持久化摘要不做召回，用于节省成本场景。volmem0 与 mem0ai 提供替代 SDK 实现。选型通过 selected_module.Memory 配置切换。

10.6 上下文源配置

系统支持 context_providers (上下文源) 配置，允许在对话前注入外部上下文 (如设备状态、环境信息)。上下文源与记忆召回协同，共同构成 LLM 的完整上下文，使对话具备场景感知能力。

十一、配置系统设计

11.1 三层配置机制

系统配置采用三层递进覆盖机制 (后层覆盖前层同名键)：

1. 基础配置：config.yaml，已提交仓库，含默认值与注释说明，是配置的基线。不应直接修改，便于随版本升级。
2. 本地覆盖：data/.config.yaml，不受版本控制，存放密钥与本地差异化配置。应用启动时必须存在，可空文件。已加入 .gitignore。
3. 远程覆盖：若 data/.config.yaml 中配置了 manager-api.url，启动时从智控台拉取远程配置覆盖本地。该层优先级最高。

重要：若使用了智控台，则本地所有 AI 相关配置均不生效，全部以智控台下发为准，实现集中运维。

11.2 配置加载器

config/config_loader.py 实现配置的递归合并：读 config.yaml → 合并 data/.config.yaml → (可选) 请求智控台 API 合并远程配置。合并采用深递归，对 dict 递归合并而非整体替换，保证局部覆盖不丢失其他默认项。结果缓存供全局读取，避免重复 IO。settings.py 的 load_config 是统一入口。

11.3 远程配置与热更新

智控台 manager-api 通过 REST 接口向本系统暴露运行时配置，使管理员在智控台调整 AI 参数 (如切换 LLM 服务商、修改系统提示词、调整 TTS 音色) 后，本系统下次拉取即生效，无需重启。update_config 方法支持运行时重新拉取配置。该设计是"智控台—语音服务"联动的关键，实现运维零停机。

11.4 配置项示例

```
# config.yaml 主要配置段
server:
  port: 8000          # WebSocket 端口
  listen_port: 8003   # HTTP 端口
  auth_key: ""        # 根密钥（优先级最高）

selected_module:
  ASR: FunASR         # ASR 类名
  TTS: EdgeTTS        # TTS 类名
  LLM: OpenAIComats   # LLM 类名
  VAD: SileroVAD      # VAD 类名
  Intent: LLMIntent   # Intent 类名
  Memory: PowerMem    # Memory 类名

close_connection_no_voice_time: 120 # 无语音超时
tts_timeout: 10                # TTS 超时
tool_call_timeout: 30          # 工具调用超时
enable_wakeup_words_response_cache: true
enable_greeting: true
```

11.5 配置校验

启动时对关键配置项校验：auth_key 有效性（非空且非占位符"你"字串）、manager-api.url 可达性、Provider 类名合法性（工厂能否导入）、MCP 端点格式（validate_mcp_endpoint）、ffmpeg 可执行（check_ffmpeg_installed）。校验失败以明确错误信息提示，避免运行时因配置错误产生不可预期的行为。

11.6 插件配置

plugins 配置段为各函数插件提供独立配置，如 get_weather 的 api_key、get_news 的 url、search_from_ragflow 的接口地址、web_search 的搜索引擎配置。插件配置与插件实现解耦，新增插件时在 config.yaml 增加对应配置段即可。

十二、关键技术设计

12.1 流式处理与首字延迟优化

本系统对实时性要求极高。全链路采用流式处理：VAD 逐帧判断、ASR 流式识别、LLM 流式生成、TTS 流式合成与句子切分。各阶段通过 asyncio 队列衔接，前一阶段产出即可被后一阶段消费，

形成流水线并发。该设计使首字延迟（用户说完到设备开始播放回复）可控制在秒级，是语音交互体验的核心指标。performance_tester 提供基准测试量化该指标。

12.2 Opus 编解码

ESP32 设备与服务端之间的音频传输采用 Opus 编码，低带宽高音质。服务端在 opus_encoder_utils 中封装 ffmpeg 编解码，业务层只处理 PCM。编码参数（采样率 16kHz、声道、比特率）可配置，适配不同网络条件。编解码通过子进程调用 ffmpeg，避免阻塞事件循环。

12.3 AEC 声学回声消除

全双工对话要求设备播放语音时仍可被用户打断。系统在 _apply_aec 中缓存近期播放的 PCM，与收音帧对齐相减消除回声。_check_aec_cache_expiry 定时清理过期缓存。该机制配合 VAD 暂停/恢复（resume_vad_detection），使打断自然流畅，是 AI 语音陪伴产品全双工体验的关键。

12.4 内容安全与输出门控

core/providers/content_safety 与 core/utils/content_safety.py 对 LLM 输出进行安全校验。系统设计了输出门控（output gate）机制：_release_gate_result、_feed_checked_output、_finish_checked_output 协同管理待输出内容的校验-释放-阻断流程。不安全内容在送入 TTS 前被拦截，保证语音陪伴产品对未成年用户的安全。门控支持按句子粒度阻断，仅截断不安全片段而不影响已正常播放的内容。

12.5 GC 管理器

语音交互产生大量临时对象（PCM 帧、Opus 包、消息体），Python 默认 GC 在长连接高并发下可能引发停顿。core/utils/gc_manager 提供独立的 GC 调度策略，在空闲期主动回收，避免在关键音频处理路径触发 GC 停顿影响实时性。gc_manager 以单例模式运行（get_gc_manager），全局统一调度。

12.6 音频速率控制

audioRateController 控制音频回传速率，匹配设备播放消费速度。若服务端产出快于设备播放，队列堆积导致延迟累积；若产出慢于播放，出现卡顿。速率控制器动态调节发送间隔，保证平滑播放。该机制对弱网场景尤为重要。

12.7 声纹识别

`core/utils/voiceprint_provider` 与 `_initialize_voiceprint` 实现说话人识别。用户首次对话登记声纹，后续对话中识别说话人身份，联动记忆与画像实现多用户区分。该能力使同一设备能服务多名家庭成员且各自保持独立记忆，是家庭场景的核心能力。

12.8 唤醒词

`core/utils/wakeup_word` 管理设备唤醒词配置，支持自定义唤醒词与离线唤醒。
`enable_wakeup_words_response_cache` 开启唤醒词响应缓存，降低重复唤醒延迟。唤醒词与声纹、VAD 协同，构成设备“听到名字即响应”的低功耗待机能力。`enable_greeting` 控制设备唤醒后是否主动问候。

12.9 对话管理

`core/utils/dialogue.py` 的 `Dialogue` 类管理多轮对话历史，支持上下文截断（防止 token 溢出）、系统提示词注入（`prompt_manager`）、few-shot 工具调用示例注入（`_inject_tool_call_fewshot`）。对话历史是 LLM 推理与记忆召回的共同上下文。`change_system_prompt` 支持运行时切换角色提示词。

12.10 提示词增强

`_init_prompt_enhancement` 与 `prompt_manager` 实现系统提示词的动态增强：根据设备绑定信息、用户画像、当前时间、记忆召回结果动态拼装 system prompt，使同一 LLM 在不同用户、不同场景下产出更贴切的回复。该机制是个性化对话的核心。

12.11 输出序号管理

`output_counter` 统一管理输出序号，为内容安全门控与速率控制提供有序基准。每个待输出句子分配唯一序号，门控按序号阻断或释放，速率控制按序号调度发送。该设计保证在并发与中断场景下输出顺序的正确性。

12.12 退出指令与通知

`exit_commands` 配置退出指令词列表，`handle_exit_intent` 插件识别后主动结束对话。
`enable_stop_tts_notify` 与 `stop_tts_notify_voice` 配置打断时的通知语音。这些细节共同打磨对话体验的边界场景。

12.13 P3 与文本工具

`core/utils/p3.py` 与 `textUtils.py` 提供文本处理工具：句子切分、标点归一化、特殊字符过滤。
`_clean_response_garbage` 清洗模型输出的垃圾文本，`_merge_tool_calls` 合并去重多工具调用。这些工具保证流水线各阶段输入输出的整洁。

十三、部署与运维设计

13.1 容器化部署

系统提供 `Dockerfile` 与 `docker-compose` 配置，支持容器化部署。`docker-compose-oceanbase.yml` 集成 OceanBase 与依赖服务，一键拉起完整运行环境。`docker-compose_all.yml` 提供全量依赖部署。生产环境推荐以容器部署保证环境一致性。`docker-compose.yml` 提供基础部署模板。

13.2 依赖管理

依赖固定在 `requirements.txt`，重型 ML 依赖（`torch 2.2.2`、`funasr 1.2.7`、`sherpa_onnx 1.12.29`、`silero_vad 6.1.0`）单独标注。推荐 Python 3.12。`requirements-dev.txt` 提供开发期额外工具。依赖安装后需校验 `ffmpeg` 可执行（`check_ffmpeg_installed` 启动强制检查）。

13.3 启动流程

1. `app.py` 入口调用 `load_config` 加载三层配置。
2. `setup_logging` 初始化 `loguru` 日志。
3. `check_ffmpeg_installed` 校验 `ffmpeg` 安装。
4. 确定 `auth_key`（配置 > 智控台 secret > 随机）。
5. 启动 `SimpleHttpServer`（端口 8003）。
6. 启动 `WebSocketServer`（端口 8000）。
7. 进入 `wait_for_exit` 等待 `SIGINT/SIGTERM` 优雅退出。
8. `monitor_stdin` 消费回车键，支持交互式命令。

13.4 日志与监控

采用 `loguru` 结构化日志，`config/logger.py` 统一配置 `log_format` 与 `log_format_file`。`audit.log` 记录关键审计事件。`WebSocket` 连接、`Provider` 初始化、对话摘要、工具调用均有日志埋点，支持按 `client_id` 追踪全链路。日志格式含时间、版本、模块、级别、文件行号，便于定位问题。

13.5 性能基准

`performance_tester/` 提供 ASR、LLM、TTS 的独立基准测试脚本，可在接入新服务商后量化首字延迟、识别准确率、合成耗时等指标，作为选型与调优依据。`performance_tester.py` 是统一入口。该机制保证系统在扩展能力矩阵时性能可量化、可对比。

13.6 优雅退出

`wait_for_exit` 通过 `asyncio.Event` 与信号处理器（SIGINT/SIGTERM）实现优雅退出。收到退出信号后，停止接收新连接，等待现有连接的 `_save_and_close` 完成，保存对话摘要，释放 Provider 资源。Windows 平台依赖 `KeyboardInterrupt` 冒泡。该设计保证退出不丢失对话上下文。

十四、安全设计

14.1 鉴权与加密

WebSocket 握手与 HTTP 接口均需鉴权。`auth_key` 作为根密钥签发 JWT，Token 带过期时间。设备密钥（`client_secret`）由智控台在设备注册时下发，连接时校验。密钥与敏感配置存于 `data/.config.yaml`（`gitignore`），不入版本控制。生产环境 `auth_key` 必须显式配置，避免随机密钥导致多实例不一致。

14.2 输入校验

对所有外部输入（WebSocket 消息、HTTP 参数、音频帧）进行校验。消息格式由 `TextMessageType` 枚举约束，非法类型拒绝处理。MCP 端点格式由 `validate_mcp_endpoint` 校验，防注入。OTA 请求校验 `device_id` 与 `version` 合法性。视觉分析请求校验图片格式与大小。

14.3 内容安全

LLM 输出经内容安全 Provider 校验，不安全内容在 TTS 前阻断。输出门控机制保证阻断不影响已正常播放的内容，仅截断后续不安全片段。该设计满足语音陪伴产品面向未成年用户的合规要求。

14.4 资源隔离

每连接独立 `ConnectionHandler`，独立队列与 Provider 实例，单连接异常不影响其他连接。队列有容量上限，防止恶意客户端耗尽内存。`clear_queues` 与 `reset_audio_states` 在连接异常时清理资源。

各类超时（close_connection_no_voice_time、tts_timeout、tool_call_timeout）防止资源被异常连接长期占用。

14.5 OTA 安全

固件包带 MD5 校验，设备下载后验证完整性，防止固件被篡改。OTA Token 由 auth_key 签名带过期，防止未授权设备拉取固件。固件分发通过 HTTPS，保证传输安全。

14.6 密钥管理

所有密钥（LLM API Key、TTS API Key、ASR API Key、auth_key、设备密钥）均存于 data/.config.yaml 或智控台远程配置，不入代码仓库。代码中通过配置读取，不硬编码。该设计满足密钥管理与轮换的合规要求。

14.7 审计日志

audit.log 记录关键安全事件：设备认证、配置拉取、OTA 请求、内容安全阻断。审计日志独立于运行日志，用于安全事件追溯。对话摘要持久化（mem_report_only 模式）也作为业务审计的一部分。

十五、数据结构设计

15.1 音频帧结构

设备与服务端之间的音频帧为二进制 WebSocket 帧，载荷为 Opus 编码数据，附带时间戳元数据。时间戳用于 AEC 对齐与速率控制。PCM 帧在服务端内部流转，采样率 16kHz、16 位单声道，是 ASR/VAD/TTS 的统一处理格式。Opus 编解码隔离在 opus_encoder_utils，业务层只感知 PCM。

15.2 TTS 句子队列结构

tts_audio_queue 的元素为三元组（sentence_type, opus_packets, text），sentence_type 取 FIRST/MIDDLE/LAST。FIRST 标记一句开始，MIDDLE 标记句中续包，LAST 标记一句结束。该三态设计使 sendAudioHandle 能按句子边界有序回传，设备可逐句播放而非整段等待。

```
# tts_audio_queue 元素结构
(sentence_type: SentenceType, # FIRST / MIDDLE / LAST
 opus_packets: list[bytes],   # Opus 编码包列表
 text: str | None)           # 对应文本（LAST 携带）
```

15.3 消息体结构

WebSocket 文本消息为 JSON，核心字段 type 标识消息类型，其余字段按类型不同。hello 消息携带设备能力、音频参数；listen 携带 state (start/stop)；iot 携带设备实体与属性；mcp 携带工具名与参数。所有消息体经 TextMessageType 枚举校验类型合法性。

```
# hello 消息示例
{
  "type": "hello",
  "device_id": "xxx",
  "client_id": "xxx",
  "audio_params": {
    "format": "opus",
    "sample_rate": 16000,
    "channels": 1
  }
}
```

15.4 对话历史结构

Dialogue 类维护对话历史为 messages 列表，每条为 {role, content} 字典。role 取 system/user/assistant/tool。系统提示词在 system 角色，用户输入在 user，LLM 回复在 assistant，工具结果在 tool。上下文截断保留首尾、裁剪中段，保证 token 不溢出且保留关键上下文。

15.5 配置数据结构

配置以嵌套 dict 表示，顶层键包括 server、selected_module、log、chat_quota、plugins、xiaozhi、context_providers、wakeup_words、mcp_endpoint 等。selected_module 的子键 ASR/TTS/LLM/VAD/Intent/Memory 为各能力的类名。三层配置递归合并为单一 dict，全局只读。

15.6 记忆存储结构

向量记忆存储为 (id, content, embedding, metadata) 四元组，embedding 为 LLM 生成的向量。知识图谱存储为 (subject, predicate, object) 三元组。两者通过 entity id 关联。用户画像为结构化 JSON，含性格、兴趣、关系等字段。所有存储由 pyobvector 客户端访问 OceanBase。

十六、异常处理设计

16.1 异常处理原则

本系统对所有外部交互（网络 IO、模型推理、工具执行）均做异常捕获与降级处理，遵循"永不因单点异常导致连接崩溃"原则。异常在最近可处理层捕获，转换为用户可理解的响应或日志记录，不向上传播导致服务退出。

16.2 Provider 异常

ASR/TTS/LLM 等 Provider 在推理失败时捕获异常，记录日志并产出降级响应（如 TTS 失败时回退默认提示音，LLM 失败时回复固定话术）。tts_timeout 与 tool_call_timeout 防止 Provider 无限阻塞。Provider 异常不中断连接，仅中断当前轮对话，下一轮自动恢复。

16.3 连接异常

WebSocket 连接断开由 handle_connection 的 try/except 捕获，触发 _save_and_close 保存对话摘要后释放资源。clear_queues 与 reset_audio_states 清理残留队列与状态。连接异常不影响其他连接，资源隔离保证系统整体稳定。

16.4 TTSException

系统定义 TTSException(RuntimeError) 专用于 TTS 合成异常。合成失败抛出 TTSException 由 chat 方法捕获，清理 TTS 队列后继续接受下一轮对话。该异常类型使 TTS 错误可与其他错误区分处理。

16.5 配额检查

_check_chat_quota 在对话前检查配额，超限则拒绝本轮对话并提示用户。chat_quota 配置段定义配额规则。该机制防止单用户过度消耗资源，保护系统整体可用性。

16.6 日志记录

所有异常均经 loguru 记录，含 client_id、异常类型、堆栈，便于按连接追踪。audit.log 单独记录安全相关异常（认证失败、内容安全阻断）。日志分级，ERROR 级以上触发告警。

十七、性能设计

17.1 首字延迟优化

首字延迟（用户说完到设备开始播放回复）是语音交互体验的核心指标。本系统通过四层优化控制延迟：流式处理（各阶段边产边消费）、句子切分（TTS 按句合成首句即播）、懒加载（Provider 按需初始化）、并发编排（ASR/LLM/TTS 协程并发）。目标首字延迟控制在秒级。

17.2 并发模型

系统基于 `asyncio` 单事件循环，所有 IO 与编排均为协程，避免线程切换开销。每连接独立事件流，互不阻塞。重型推理（本地 ASR/TTS/LLM）通过线程池与事件循环协同，防止阻塞主循环。`_report_worker` 等后台协程在空闲期执行非关键任务。

17.3 内存优化

GC 管理器在空闲期主动回收，避免在音频处理路径触发停顿。队列设容量上限防止内存膨胀。Provider 实例按 `client_id` 缓存复用，避免重复加载模型。Opus 包用后即释，PCM 帧处理完即丢弃。`close_connection_no_voice_time` 自动关闭空闲连接释放资源。

17.4 网络优化

Opus 编码低带宽传输，16kHz 单声道下约 24kbps。音频速率控制匹配设备消费速度，避免缓冲溢出。WebSocket 长连接复用，避免频繁握手。`enable_websocket_ping` 可选开启心跳保活，适配 NAT 超时场景。

17.5 性能基准测试

`performance_tester/` 提供独立基准测试，量化各 Provider 的延迟与吞吐。ASR 测试统计识别延迟与准确率，LLM 测试统计首 token 延迟与生成速度，TTS 测试统计首音频包延迟与合成速度。测试结果作为服务商选型与参数调优的量化依据。`performance_tester.py` 为统一入口，支持按配置批量测试。

17.6 启动性能

Provider 懒加载使启动不等待全部模型加载，首连接到达时按需初始化。`_background_initialize` 在后台初始化重型 Provider，不阻塞首帧处理。该设计使系统冷启动到可服务状态时间可控。

十八、接口协议详细设计

18.1 WebSocket 握手协议

设备通过 WebSocket 连接 `ws://<host>:8000/`，握手时在 `query` 或 `header` 携带 JWT 或 OAuth-Token。WebSocketServer._handle_auth 校验 Token，通过后创建 ConnectionHandler 并进入 handle_connection 主循环。握手失败连接由 _http_response 返回 401。

18.2 hello 握手消息

连接建立后设备发送 hello 消息，携带 device_id、client_id、audio_params (format、sample_rate、channels)。服务端响应 hello 确认会话建立，下发 session_id 与服务端音频参数。hello 消息由 helloHandle 处理，完成设备能力协商。

18.3 listen 监听控制

listen 消息的 state 字段控制收音：start 表示设备开始上传音频帧，stop 表示停止。服务端据此启停 ASR 流水线。该消息也用于设备主动告知"我说完了"，配合 VAD 静音判定共同确认语音段边界。

18.4 iot 设备控制协议

iot 消息携带设备实体 (entity) 与属性 (attributes)，支持上报与控制双向。设备上报状态时 from=device，服务端控制时 from=server。LLM 可通过工具调用触发 iot 控制消息，实现语音控制硬件。

18.5 OTA 接口协议

OTA 端点 `/xiaozhi/ota/` 接受 GET 请求，query 参数 device_id 与 version。响应 JSON 含 version (最新版本)、size (字节数)、md5 (校验和)、url (下载地址)。设备按 url 下载 .bin 文件，校验 md5 后升级。Token 通过 query 或 header 携带，由 auth_key 签名。

18.6 视觉分析接口协议

视觉端点 `/mcp/vision/explain` 接受 POST，body 含 image (Base64 或 URL) 与可选 prompt。响应 JSON 含 description (图像描述)。JWT 在 Authorization header 携带。该端点作为 MCP 工具被对话调用，实现多模态交互。

18.7 错误响应

HTTP 错误响应统一 JSON 格式：{code, message}。code 为业务错误码，message 为人类可读说明。鉴权失败返回 401，参数错误返回 400，服务异常返回 500。错误消息不泄露敏感信息（密钥、内部路径）。

十九、设备生命周期管理设计

19.1 设备注册

设备首次接入前需在智控台完成注册，获取 device_id 与 client_secret。注册信息含设备型号、MAC 地址、归属用户。智控台将注册信息持久化并通过 manager-api 暴露给本系统校验。设备出厂后通过 OTA 绑定服务地址，首次上电即向本系统发起 WebSocket 连接完成激活。

19.2 设备激活与绑定

设备首次连接时携带 device_id 但未绑定用户，进入待绑定流程。
_discard_message_with_bind_prompt 引导用户完成绑定，绑定后用户与设备建立归属关系。绑定信息由智控台管理，本系统在连接时拉取校验。已绑定设备直接进入对话流程。

19.3 连接保活

WebSocket 长连接通过 ping 消息保活（enable_websocket_ping 可选）。设备定期发送 ping，服务端响应确认存活。close_connection_no_voice_time（默认 120 秒）无语音输入即关闭连接，释放资源。连接断开后设备自动重连，本系统按 client_id 复用 Provider 实例。

19.4 设备重启

handle_restart 处理设备主动重启请求，清理当前连接状态后重新初始化。_init_connection_state 重置连接级状态，_save_and_close 保存对话摘要后关闭。重启不等同于断连，重启保留设备绑定与记忆，仅重置会话状态。

19.5 设备解绑

设备解绑由智控台发起，解除用户与设备的归属关系。解绑后设备进入待绑定流程，原有用户的记忆与画像保留但不再对该设备生效。本系统在连接时拉取最新绑定状态，智控台解绑即时生效。

19.6 固件升级

OTA 固件升级是设备生命周期的重要环节。设备请求 `/xiaozhi/ota/` 获取最新固件元数据，按 url 下载 .bin，校验 md5 后升级。升级失败设备保持原版本，不影响服务。固件版本由智控台管理，支持按设备型号灰度发布。

19.7 设备状态上报

设备通过 `report` 消息上报运行状态：电量、网络强度、音量、温度等。`reportHandle` 接收，`_report_worker` 后台消费，`_process_report` 写入日志与智控台。状态数据用于运维监控与故障预警，是设备生命周期管理的数据基础。

二十、可观测性设计

20.1 日志体系

系统采用 `loguru` 结构化日志，`config/logger.py` 统一配置。`log_format` 含时间、版本、模块、tag、级别、文件行号、消息，便于按维度检索。`log_format_file` 为文件持久化格式。日志级别分 `DEBUG/INFO/WARNING/ERROR/CRITICAL`，生产环境默认 `INFO`。日志支持按 `client_id` 追踪单设备全链路，便于问题定位。

20.2 审计日志

`audit.log` 独立于运行日志，记录关键安全与业务事件：设备认证、配置拉取、OTA 请求、内容安全阻断、配额超限。审计日志不可篡改，用于安全事件追溯与合规审计。对话摘要持久化（`mem_report_only` 模式）也作为业务审计的一部分。

20.3 对话摘要

每轮对话结束后 `_log_dialogue_summary` 生成摘要，含用户输入、系统回复、工具调用、意图命中情况。摘要写入记忆系统与日志，既是记忆召回的数据源，也是对话质量分析的数据基础。摘要不记录原始音频，仅记录文本与元数据。

20.4 性能埋点

关键路径埋点统计延迟：VAD 判定延迟、ASR 识别延迟、LLM 首 token 延迟、TTS 首包延迟、首字端到端延迟。埋点数据写入日志，可聚合分析瓶颈环节。`performance_tester` 提供离线基准，运行时埋点提供在线指标，两者互补。

20.5 健康检查

系统可对接智控台健康检查接口，上报服务存活、Provider 可用性、连接数、队列积压等指标。智控台据此做运维告警与负载均衡。健康数据是系统可观测性的实时视图。

20.6 错误追踪

异常经 loguru 记录完整堆栈，含 client_id、异常类型、发生位置。ERROR 级以上触发告警。可对接 Sentry 等错误追踪平台（通过 loguru sink）实现错误的聚合与通知。错误追踪保证线上问题可被发现与定位。

二十一、数据流详细设计

21.1 收音数据流

设备上传 Opus 帧经 WebSocket 到达 handle_connection，_route_message 识别为二进制后调用 _process_websocket_audio。_decode_opus_packet 将 Opus 解为 PCM，_apply_aec 做回声消除，_put_asr_audio_with_aec 将 PCM 帧入 asr_audio_queue。同时 receiveAudioHandle 调用 vad.is_vad 判定语音活动，触发 ASR 开始/结束。

```
Opus 帧 → _process_websocket_audio → _decode_opus_packet
→ _apply_aec → _put_asr_audio_with_aec → asr_audio_queue
→ ASR Provider 流式识别
```

21.2 识别数据流

ASR Provider 从 asr_audio_queue 取 PCM 帧流式识别，VAD 判定静音时触发产出最终文本。文本经 handle_user_intent 意图识别：命中则直接执行工具产出响应，未命中则进入 chat 方法。识别数据流的关键是 VAD 与 ASR 的解耦协作——VAD 切段，ASR 转写。

21.3 对话数据流

chat 方法接收识别文本，构造 messages（system prompt + 对话历史 + 当前输入），调用 LLM Provider 流式生成。生成期间若触发工具调用，_handle_function_result 暂停文本送入 TTS，执行工具后结果回灌 LLM 继续生成。生成的文本流式送入 TTS Provider。

```
识别文本 → chat → 构造 messages
→ LLM 流式生成 → 触发工具?
是: _handle_function_result → 回灌 → 继续生成
否: 文本 → TTS Provider
```

21.4 合成数据流

TTS Provider 将 LLM 流式文本按标点切分为句子，逐句合成 Opus 包，按 FIRST/MIDDLE/LAST 标记入 `tts_audio_queue`。`sendAudioHandle` 从队列取包，按句子序号有序回传设备。`audioRateController` 调节回传速率匹配设备播放速度。合成数据流的关键是流式与句子切分，使首句合成完成即开始播放。

21.5 记忆数据流

对话结束后 `_log_dialogue_summary` 生成摘要，Memory Provider 提取记忆事实向量化存入 OceanBase，用户画像由 UserMemory 提取存入知识图谱。下一轮对话前 `_initialize_memory` 按当前输入召回相关记忆与画像，经 `prompt_manager` 增强进 `system prompt`。记忆数据流的双向（存入与召回）构成跨会话个性化。

21.6 工具调用数据流

LLM 决定调用工具时产出 `tool_calls`，UnifiedToolHandler 按工具名分发到对应执行器（`device_iot/device_mcp/mcp_endpoint/server_mcp/server_plugins`）。执行器执行工具，结果经 `_handle_function_result` 构造为 `tool` 消息回灌 LLM。`_merge_tool_calls` 合并去重多工具调用保证幂等。工具调用数据流是 LLM 能力扩展的核心。

21.7 上报数据流

设备上报状态经 `reportHandle` 接收，入上报队列，`_report_worker` 后台消费，`_process_report` 写入日志与智控台。上报数据流与对话数据流隔离，互不阻塞，保证上报不影响对话实时性。

二十二、扩展机制设计

22.1 Provider 扩展

新增 ASR/TTS/LLM/VAD/Intent/Memory 服务商的步骤：在 `core/providers/<能力>/` 下新增实现文件，实现继承基类的类，在 `data/.config.yaml` 或智控台 `selected_module` 配置类名。工厂函数 `create_instance` 动态导入并实例化，无需改业务代码。该机制使能力矩阵可持续扩展，目前已支持 ASR 14 家、TTS 20 家、LLM 11 家。

22.2 函数插件扩展

新增工具能力的步骤：在 `plugins_func/functions/` 下新增模块，实现 `BaseFunction` 子类并用 `@register_function(name, desc, type)` 装饰。`loadplugins.auto_import_modules` 启动时自动扫描注册，LLM 即可调用。该机制使工具能力可由开发者独立扩展，目前已内置 15 个插件。

22.3 消息类型扩展

新增 WebSocket 消息类型的步骤：在 `textMessageType` 枚举增加成员，在 `textHandler` 下新增处理器，在 `TextMessageHandlerRegistry` 注册分发。新类型立即对设备可用，不影响既有类型处理。该机制使设备协议可持续演进。

22.4 上下文源扩展

`context_providers` 配置支持注入外部上下文。新增上下文源只需实现接口并在配置声明，对话前自动拉取注入 `system prompt`。该机制使系统能接入业务系统数据（如设备状态、环境信息）作为对话上下文。

22.5 MCP 工具扩展

MCP 协议支持动态接入第三方工具集。在配置声明 MCP 端点，`validate_mcp_endpoint` 校验后，端点暴露的工具自动纳入 `UnifiedToolHandler` 对 LLM 可用。该机制使系统能接入广泛的 MCP 生态工具而不改动核心代码。

22.6 开闭原则

上述四类扩展（Provider、插件、消息、上下文）均遵循开闭原则：对扩展开放（新增实现类/模块/枚举），对修改关闭（不改工厂、调度器、分发器核心逻辑）。该设计保证系统在功能持续扩展的同时核心稳定，降低维护成本与回归风险。

二十三、容量与配额设计

23.1 对话配额

`chat_quota` 配置段定义对话配额规则，`_check_chat_quota` 在每轮对话前检查。配额可按用户、按设备、按时间段限制对话次数或 token 消耗，防止单用户过度占用资源。超限拒绝本轮对话并提示用户。该机制是系统多租户资源公平的基础。

23.2 连接数控制

系统支持并发多连接，每连接独立 `ConnectionHandler`。连接数受服务端资源（内存、文件描述符）约束。`close_connection_no_voice_time` 自动关闭空闲连接释放资源。生产环境通过多实例部署与负载均衡承载更高并发。

23.3 队列容量

`asr_audio_queue` 与 `tts_audio_queue` 设容量上限，防止异常客户端或慢消费导致内存膨胀。队列满时丢弃最旧帧（音频实时性高于完整性）或阻塞生产者（视队列语义而定）。该设计保证系统在异常输入下的内存安全。

23.4 超时保护

三类超时共同保护资源不被异常占用：`close_connection_no_voice_time`（120s 无语音关闭连接）、`tts_timeout`（10s TTS 合成超时）、`tool_call_timeout`（30s 工具调用超时）。超时触发清理与降级，保证系统在异常 Provider 或恶意客户端下的稳定。

23.5 delete_audio 配置

`delete_audio` 配置控制是否在处理删除临时音频文件，平衡磁盘占用与可追溯性。生产环境通常开启以控制磁盘，调试期可关闭以保留样本。该配置体现系统对资源占用的细粒度控制。

23.6 资源隔离

每连接独立队列与 Provider 实例，单连接异常（队列堆积、Provider 崩溃）不扩散至其他连接。连接级资源在 `close` 时统一清理。该隔离保证系统整体可用性不受单连接异常影响，是多租户安全的关键。

二十四、插件系统详细设计

24.1 插件架构

`plugins_func/` 是系统的工具能力扩展层，由 `loadplugins.py`、`register.py` 与 `functions/` 三部分构成。`loadplugins.auto_import_modules("plugins_func.functions")` 在启动时动态扫描 `functions/` 目录下所有 `.py` 模块并导入，触发模块级的 `@register_function` 装饰器执行，将能力注册至全局注册表。该设计实现零配置发现——新增插件只需新增文件。

24.2 注册机制

`@register_function(name, desc, type)` 装饰器接收三个参数：`name` 为工具唯一标识（LLM 据此调用），`desc` 为自然语言描述（LLM 据此理解工具用途），`type` 为 `ToolType` 枚举（`SYS_PLAYER/SENSOR` 等，用于分类与权限）。装饰器将被装饰类的引用存入全局注册表，`UnifiedToolHandler` 启动时从注册表加载全部工具对 LLM 暴露。

```
@register_function(name="get_weather", desc="查询指定城市天气",
                  type=ToolType.SENSOR)
class GetWeatherFunction(BaseFunction):
    async def execute(self, city: str, **kwargs):
        # 调用天气 API, 返回结构化结果
        return {"city": city, "weather": "...", "temp": "..."}
```

24.3 插件基类

所有插件继承 `BaseFunction`，实现 `execute` 方法。`execute` 接收关键字参数（由 LLM 从工具调用参数映射），返回结构化结果。结果经 `_handle_function_result` 回灌 LLM，LLM 据此生成自然语言回复。`BaseFunction` 提供日志、异常捕获、配额检查的通用支撑，插件只需关注业务逻辑。

24.4 插件配置

各插件可在 `config.yaml` 的 `plugins` 段配置独立参数：`get_weather` 的 `api_key`、`get_news` 的 `url` 列表、`search_from_ragflow` 的接口地址与 `key`、`web_search` 的搜索引擎选择。插件在 `execute` 时读取配置，配置缺失则降级或提示。该设计使插件能力可运维化调整而无需改代码。

24.5 Home Assistant 集成

系统提供 `hass_init`、`hass_get_state`、`hass_set_state`、`hass_play_music` 四个插件对接 Home Assistant 智能家居平台。`hass_init` 初始化连接，`get/set_state` 读写设备状态，`play_music` 通过 HA 控制媒体播放器。该集成使本系统能作为语音中枢控制 HA 生态设备，扩展了智能家居场景能力。

24.6 退出与角色插件

`handle_exit_intent` 插件识别退出指令（`exit_commands` 配置），主动结束对话。`change_role` 插件支持运行时切换智能体角色（`change_system_prompt`）。`call_device` 插件支持呼叫其他设备。这些插件覆盖了对话控制的边界场景，使系统能响应用户的元指令（退出、换角色）而非仅业务问答。

24.7 知识库与搜索插件

`search_from_ragflow` 插件对接 RAGFlow 知识库，支持私有知识检索增强生成。`web_search` 插件支持联网搜索（多搜索引擎可选），使 LLM 能获取实时信息。两个插件分别覆盖私有知识与公网信息两个维度，是 RAG（检索增强生成）能力的实现基础。

二十五、会话管理设计

25.1 会话标识

每个 `WebSocket` 连接对应一个独立会话，由 `client_id` 唯一标识。会话状态封装在 `ConnectionHandler` 实例中，包括对话历史（`Dialogue`）、`Provider` 实例、队列、认证态。会话与设备多对一：同一设备断连重连视为新会话，但按 `client_id` 复用 `Provider` 与记忆，保证体验连续。

25.2 对话上下文

`Dialogue` 类管理多轮对话上下文，维护 `messages` 列表。上下文截断策略保留首条 `system prompt` 与最近若干轮，裁剪中段历史，防止 `token` 溢出。`_inject_tool_call_fewshot` 在上下文注入工具调用示例，提升 LLM 工具调用准确率。`prompt_manager` 与 `_init_prompt_enhancement` 动态增强 `system prompt`，注入用户画像、记忆召回、当前时间等。

25.3 会话恢复

设备断连重连后，本系统按 `client_id` 复用已初始化的 `Provider` 实例，避免重复加载模型。对话历史从记忆系统召回最近摘要重建上下文，使会话在断连后仍能保持上下文连续。该设计保证网络抖动下的体验连续性。

25.4 多用户会话

声纹识别使同一设备能区分多用户会话。不同说话人对应不同的记忆与画像，`_initialize_memory` 按声纹识别结果加载对应用户的记忆。该设计支持家庭场景——同一设备服务多名家庭成员，各自独立记忆与画像，互不干扰。

25.5 会话终止

会话终止触发 `_save_and_close`：生成对话摘要写入记忆系统，清理队列与状态，释放 `Provider` 引用（不销毁实例以便复用）。终止可由设备主动（`close` 消息）、超时（无语音超时）或异常（连接断开）触发。终止后设备可重连开启新会话。

25.6 问候与唤醒

`enable_greeting` 控制设备唤醒后是否主动问候。问候语由智能体角色配置决定，经 TTS 合成播放。唤醒词响应缓存（`enable_wakeup_words_response_cache`）缓存常见唤醒词的响应，降低重复唤醒延迟。问候与唤醒是会话起始的用户体验细节。

二十六、设备能力协商设计

26.1 能力协商协议

设备在 `hello` 握手消息中声明自身能力：音频参数（`format`、`sample_rate`、`channels`）、支持的 IoT 实体、唤醒词。服务端据此协商后续交互参数。协商结果在 `hello` 响应中回传设备，双方按协商结果通信。该机制保证异构设备（不同硬件能力）能接入同一服务端。

26.2 音频参数协商

设备声明支持的音频格式与采样率，服务端选择双方支持的参数。默认 Opus 16kHz 单声道。若设备不支持 Opus，服务端可降级为 PCM 或其他编码。协商结果影响 Opus 编解码参数与 ASR/TTS 的采样率配置。

26.3 IoT 实体协商

设备声明拥有的 IoT 实体（传感器、开关、灯等），服务端据此注册可控制设备列表。LLM 通过工具调用控制这些实体。设备能力变化时通过 `iot` 消息上报更新。该协商使 LLM 只感知设备实际拥有的能力，避免无效控制指令。

26.4 唤醒词协商

设备声明支持的唤醒词，服务端在 `wakeup_words` 配置中匹配。唤醒词用于设备本地唤醒，触发后建立连接进入会话。自定义唤醒词通过智控台下发给设备。唤醒词是设备"听到名字即响应"的触发条件。

26.5 OTA 地址下发

设备首次连接时，服务端在 OTA 响应中下发 WebSocket 地址与 MQTT 网关配置，使设备出厂后绑定服务地址。该机制使设备无需预置服务地址，支持出厂后动态绑定与迁移。`mqtt_gateway` 与 UDP 网关配置也通过 OTA 下发。

二十七、模型管理设计

27.1 模型文件管理

本地推理模型（如 SenseVoiceSmall ASR 模型）存放于 `models/` 目录。模型文件不入版本控制（体积大），由部署时下载或挂载。模型路径在 `Provider` 配置中声明，加载时按路径读取。该设计使模型可独立于代码更新。

27.2 模型懒加载

本地模型加载耗时较长，系统通过 `_background_initialize` 在后台协程加载，不阻塞首帧处理。加载完成后 `Provider` 实例按 `client_id` 缓存复用。首次连接的模型加载延迟由性能基准量化，是冷启动优化的重点。

27.3 模型切换

切换模型只需修改 `selected_module` 配置的类名，工厂函数动态导入新实现。云端模型切换只需改 API Key 与 `endpoint` 配置。智控台远程配置使模型切换无需重启本系统。该设计支持 A/B 测试与服务商灰度迁移。

27.4 `system_prompt` 管理

`core/providers/llm/system_prompt.py` 提供系统提示词的动态管理。智能体角色的提示词由智控台配置下发，运行时可切换（`change_system_prompt`）。`_init_prompt_enhancement` 在基础提示词上叠加用户画像、记忆召回、当前时间等增强信息，使同一智能体在不同用户/场景下表现差异化。

27.5 多模态模型

`vllm` `Provider`（`core/providers/vllm`）支持本地多模态推理，用于视觉分析端点的图像理解。`qwen3_asr_flash` 等多模态 ASR 支持语音+视觉联合理解。多模态能力扩展了系统 beyond 纯语音的交互边界。

二十八、版本与兼容性设计

28.1 软件版本

本系统版本 V1.0。版本号在日志格式中嵌入（`log_format` 的 `version` 字段），便于运行时识别部署版本。版本管理遵循语义化版本，主版本号变更表示不兼容的架构调整。

28.2 设备固件兼容

OTA 端点按设备型号下发对应固件，支持多型号并存。固件版本号在 OTA 响应中返回，设备校验版本号决定是否升级。本系统兼容多版本固件设备同时接入，消息处理按设备能力协商结果适配。

28.3 配置兼容

`config.yaml` 随版本演进新增配置项，旧 `data/.config.yaml` 缺失的键由 `config.yaml` 默认值补齐（递归合并）。该设计保证配置文件跨版本兼容，升级不破坏既有本地配置。

28.4 协议兼容

WebSocket 消息协议通过 `TextMessageType` 枚举约束，新增类型不影响旧设备（旧设备不发送新类型，服务端忽略未知类型或回退默认处理）。该前向兼容设计使设备固件与服务端可独立演进。

28.5 上游同步

本系统基于上游 `xiaozhi-esp32-server` 二次开发，上游 MIT 协议允许修改与分发。本系统在上游基础上新增独创代码，新增代码著作权归属上海爱予慧。上游 LICENSE 随发行物保留，符合 MIT 要求。本系统不回吐上游，保持独立演进。

二十九、MQTT 网关集成设计

29.1 网关架构

除 WebSocket 直连外，系统支持通过 MQTT 网关接入设备。`mqtt-gateway` 作为桥接层，将局域网设备的 MQTT/UDP 协议转换为服务端可处理的格式。该设计覆盖非 WebSocket 组网场景，如经网关转发的局域网设备或受限网络环境。网关与设备协议由 `mqtt_gateway` 的 `.env` 配置，格式为 `host:port`。

29.2 MQTT 音频桥接

`_process_mqtt_audio_message` 处理来自 MQTT 通道的音频消息，与 WebSocket 音频统一汇入 `_put_asr_audio_with_aec` 进入 ASR 流水线。该设计保证无论设备经何种通道接入，下游处理一致。MQTT 签名密钥用于生成 MQTT 连接密码，保证网关接入鉴权。

29.3 UDP 网关

系统支持 UDP 网关配置，用于低延迟或受限协议场景。UDP 网关同样经 mqtt-gateway 桥接转换。网关配置通过 OTA 下发设备，使设备出厂后绑定网关地址。多网关支持使系统能适配复杂网络拓扑。

29.4 网关鉴权

MQTT 网关接入需鉴权，签名密钥由 mqtt_gateway 的 .env 配置。服务端校验网关消息的签名合法性，拒绝未授权网关。该机制保证网关接入安全，防止恶意网关注入伪造音频。

三十、内容安全详细设计

30.1 安全分层

系统内容安全分输入侧与输出侧两层。输入侧：用户语音识别文本在进入 LLM 前不做内容过滤（保留用户表达自由），但对话摘要写入记忆时可经安全过滤。输出侧：LLM 生成的文本在送入 TTS 前经内容安全 Provider 校验，不安全内容阻断。该分层平衡了表达自由与输出安全。

30.2 输出门控机制

输出门控（output gate）是本系统的独创安全机制。_release_gate_result、_feed_checked_output、_finish_checked_output 三方法协同管理待输出内容的校验-释放-阻断流程。LLM 流式产出的文本先进入门控缓冲，经安全校验通过后释放至 TTS；不安全片段在门控截断，不影响已释放并播放的内容。该设计实现按句子粒度的精准阻断。

```
LLM 流式文本 → _feed_checked_output(进入门控缓冲)
→ 内容安全校验
通过 → _release_gate_result(释放至 TTS)
不通过 → 截断该句，继续下一句
→ _finish_checked_output(本轮结束)
```

30.3 内容安全 Provider

core/providers/content_safety 与 core/utils/content_safety.py 实现安全校验。支持基于关键词、基于分类模型、基于 LLM 判断的多种策略。校验结果含是否安全与不安全原因，用于日志与审计。Provider 可按配置切换策略，适配不同合规要求。

30.4 安全审计

所有内容安全阻断事件写入 `audit.log`，含 `client_id`、阻断内容、阻断原因、时间。审计日志用于合规追溯与策略调优。阻断不向用户暴露原始判定细节，仅以中性话术回退，避免提示用户绕过。

30.5 边界场景

`safety_context` 在 `chat` 方法中传递安全上下文，记录当前对话的安全相关状态（如已阻断次数）。`output_gate` 参数允许调用方指定自定义门控策略。`_log_dialogue_summary` 在摘要中标注安全事件，使记忆召回时感知历史安全上下文。这些边界设计保证安全机制与对话流程的深度集成。

三十一、部署架构设计

31.1 单实例部署

开发与小规模生产采用单实例部署：一个 `app.py` 进程同时承载 WebSocket (8000) 与 HTTP (8003) 双服务，OceanBase 与 Redis 通过 `docker-compose-oceanbase.yml` 拉起。该部署适合中小规模（百级并发连接），资源占用低，运维简单。

31.2 多实例部署

大规模生产采用多实例部署：多个 `app.py` 进程经负载均衡分发 WebSocket 连接，OceanBase 与 Redis 独立集群。由于连接状态在进程内（`ConnectionHandler`），负载均衡需按 `client_id` 会话保持（sticky session），保证设备重连命中同一实例复用 Provider。HTTP 服务无状态可任意分发。

31.3 容器编排

Dockerfile 定义镜像构建，`docker-compose` 编排服务。生产环境推荐 Kubernetes 编排，支持滚动更新与自愈。镜像分层：基础 Python 镜像 + 依赖层 + 代码层，依赖层缓存加速构建。重型 ML 依赖（`torch/funasr`）单独分层，利用缓存。

31.4 数据层部署

OceanBase 作为记忆向量与图谱存储，生产环境独立部署或集群。OceanBase 兼容 MySQL，通过 `pyobvector` 客户端访问。Redis 作为缓存，存储热配置与连接态。数据层与应用层分离，支持独立扩缩容与备份。

31.5 网络部署

WebSocket (8000) 与 HTTP (8003) 对外暴露，经反向代理 (nginx) 终结 TLS 与负载均衡。内部 OceanBase、Redis、智控台通信走内网。mqtt-gateway 独立部署于设备侧网络，桥接局域网设备。网络分层保证对外仅暴露必要端口。

31.6 配置管理

生产环境配置通过智控台远程下发，data/.config.yaml 仅存 manager-api.url 与最小本地配置。该设计使多实例配置一致，运维集中化。配置变更通过智控台操作，本系统运行时拉取生效，无需逐实例重启。

三十二、灾备与高可用设计

32.1 无状态服务

WebSocket 与 HTTP 服务本身无状态（连接态在进程内但不跨实例共享），任一实例故障不影响其他实例。负载均衡健康检查自动剔除故障实例，新连接路由至健康实例。该设计保证服务层高可用。

32.2 记忆数据持久化

记忆与图谱存储于 OceanBase，对话摘要持久化。即使服务实例全部故障，记忆数据不丢失，恢复后可重建上下文。OceanBase 自身支持高可用部署（多副本）。该设计保证用户长期记忆的安全。

32.3 Provider 降级

单一 Provider 故障（如某云 ASR 不可用）不导致服务中断——可切换至备用 Provider（修改 selected_module）。多服务商矩阵（ASR 14 家、TTS 20 家、LLM 11 家）提供充分冗余，任一服务商故障可快速切换。该设计是服务可用性的能力层保障。

32.4 优雅退出与恢复

实例退出时 wait_for_exit 保证现有连接 _save_and_close 完成，不丢失对话摘要。实例恢复后按 client_id 复用记忆与 Provider，设备重连即恢复服务。该设计保证计划内运维（如升级）的用户无感。

32.5 超时保护

三类超时（无语音关闭、TTS 超时、工具调用超时）防止异常连接或 Provider 阻塞导致资源耗尽。即使发生故障（如 LLM 服务卡死），超时机制保证系统不无限等待，自动降级恢复。该设计是故障隔离的底层保障。

三十三、数据隐私设计

33.1 音频数据处理

用户音频在服务端以 PCM/Opus 形式临时处理，处理后即丢弃（delete_audio 控制是否留存临时文件）。音频不入版本控制、不持久化长期存储。识别文本进入对话历史与记忆，但原始音频不保留。该设计最小化音频数据留存，符合隐私最小化原则。

33.2 记忆数据隔离

记忆与画像按 client_id（设备）与声纹（用户）隔离存储。不同设备、不同用户的记忆互不可见。_initialize_memory 按当前用户身份加载对应记忆，绝不跨用户召回。该隔离保证多用户场景下的隐私边界。

33.3 对话摘要脱敏

_log_dialogue_summary 生成的对话摘要写入记忆与日志。摘要为文本元数据，不含原始音频。敏感信息（如密钥、身份证号）在摘要中不刻意保留，依赖 LLM 生成自然摘要的特性。审计日志记录阻断事件但不暴露原始不安全内容细节。

33.4 密钥与凭证

所有密钥（API Key、auth_key、设备密钥）存于 data/.config.yaml 或智控台远程配置，不入代码仓库与版本控制。日志不记录密钥明文。该设计保证凭证安全，符合密钥管理合规要求。

33.5 传输安全

WebSocket 与 HTTP 对外经反向代理终结 TLS，保证传输加密。OTA 固件通过 HTTPS 下载。内部服务间通信走内网。该设计保证数据传输的机密性与完整性。

33.6 用户数据权利

用户可通过智控台管理对话记忆与画像数据（查看、导出、删除）。设备解绑后用户记忆保留但不再对该设备生效。该设计支持用户对自身数据的管理权利，符合数据保护要求。

三十四、运维自动化设计

34.1 配置自动化

智控台集中管理智能体配置、模型参数、设备注册，本系统运行时自动拉取生效。运维人员无需登录服务器修改配置文件，全部通过智控台操作。该设计降低运维成本与人为错误风险，实现配置即代码。

34.2 日志集中化

loguru 日志可对接集中化日志平台（通过 sink）。日志含 client_id、版本、模块等结构化字段，便于按维度检索分析。错误日志可对接告警平台实现自动通知。该设计支持大规模部署下的可观测性。

34.3 健康监控

系统可对接智控台健康检查，上报服务存活、连接数、队列积压、Provider 可用性。智控台据此做告警与负载均衡决策。健康数据是运维自动化的实时输入。

34.4 容器编排运维

容器化部署支持滚动更新（零停机升级）与自愈（故障实例自动重启）。镜像分层与依赖缓存加速构建发布。该设计使部署运维标准化、自动化，降低人为干预。

34.5 性能基准自动化

performance_tester 可纳入 CI/CD 流水线，在接入新 Provider 或配置变更后自动运行基准测试，对比指标变化。该机制使性能回归可被自动发现，保证演进中的性能可控。

34.6 OTA 灰度发布

固件 OTA 支持按设备型号灰度发布，逐步放量升级。灰度发布降低固件缺陷的全局影响，支持回滚。该设计使设备固件演进的风险可控。

三十五、数字人集成设计

35.1 数字人定位

本系统可与 digital-human（数字人）项目集成，模拟 ESP32 终端设备用于测试与演示。数字人通过 WebSocket 连接本系统，复用完整语音交互流水线。该集成使开发期可在无实体设备情况下验证服务端逻辑，降低测试门槛。

35.2 测试入口

test/ 目录提供基于浏览器的 WebSocket 测试页面，可手动触发对话、上传音频、查看识别与合成结果。该入口是开发期调试的主要手段，支持快速验证流水线各环节。数字人与浏览器测试页共同构成无设备测试矩阵。

35.3 演示场景

数字人可承载视觉表现（口型、表情）同步语音输出，用于产品演示。本系统提供语音交互后端能力，数字人提供前端表现，两者经 WebSocket 解耦协作。该集成使演示场景无需 ESP32 实体硬件即可展现完整产品体验。

35.4 协议复用

数字人与实体设备复用同一 WebSocket 协议与消息类型，使本系统无需区分终端类型即可服务。该设计保证测试终端与生产终端的协议一致，测试结论可迁移至生产。

三十六、系统初始化设计

36.1 初始化序列

app.py 启动初始化严格按依赖顺序执行：setup_logging 先就绪保证后续日志可记录；load_config 加载三层配置供后续模块读取；check_ffmpeg_installed 校验音频编解码依赖；确定 auth_key 供鉴权用；启动 SimpleHttpServer 与 WebSocketServer；进入 wait_for_exit 主循环。序列保证每步前置依赖已就绪，避免初始化竞态。

36.2 Provider 懒初始化

Provider 不在启动时初始化，而在首连接由 _initialize_components 按需创建。
_background_initialize 协程在后台初始化重型 Provider（如本地 ASR 模型加载），不阻塞首帧处理。

`modules_initialize.py` 是统一初始化入口，按依赖顺序（VAD→ASR→LLM→TTS→Intent→Memory）初始化各能力。该设计使冷启动到可服务时间可控。

36.3 连接级初始化

每连接建立后执行连接级初始化：`_initialize_components` 创建 `Provider` 实例（或复用缓存）、`_initialize_memory` 加载记忆、`_initialize_intent` 初始化意图、`_init_prompt_enhancement` 增强提示词、`_inject_tool_call_fewshot` 注入工具示例。`_init_connection_state` 重置连接态。这些初始化使连接就绪可对话。

36.4 报告线程初始化

`_init_report_threads` 初始化上报相关后台协程，包括 `_report_worker` 消费上报队列、`_check_timeout` 检测超时连接、`_check_aec_cache_expiry` 清理 AEC 缓存。这些后台协程与主对话流程并发，互不阻塞，共同维护连接健康。

36.5 失败处理

初始化失败（如配置缺失、依赖未装、模型未下载）以明确错误信息提示并终止启动，避免带病运行。`check_ffmpeg_installed` 失败直接退出。`Provider` 类名非法由工厂抛 `ImportError`。`fail-fast` 设计保证系统只在健康状态下服务。

三十七、系统初始化与运行时协同设计

37.1 启动期与运行时协同

启动期完成无依赖可执行的初始化（日志、配置、依赖校验、双服务启动），运行期按需完成重依赖初始化（`Provider` 懒加载、记忆召回）。该划分使启动快速（秒级），重初始化延后到首连接或后台，不阻塞服务就绪。`update_config` 在运行期支持重新拉取配置，实现热更新。

37.2 后台协程与主循环

主事件循环承载 `WebSocket` 与 `HTTP` 服务的连接处理，后台协程（`_background_initialize`、`_report_worker`、`_check_timeout`、`_check_aec_cache_expiry`、`monitor_stdin`）并发运行不阻塞主循环。协程间通过共享状态（`ConnectionHandler`）与队列协作。该设计是 `asyncio` 单循环并发模型的核心。

37.3 信号处理

`wait_for_exit` 注册 `SIGINT/SIGTERM` 信号处理器，设置 `stop_event` 触发优雅退出。退出时停止接受新连接，等待现有连接 `_save_and_close`，保存对话摘要，释放资源。Windows 依赖 `KeyboardInterrupt` 冒泡。该设计支持容器编排的优雅终止信号（`SIGTERM`）。

37.4 `monitor_stdin`

`monitor_stdin` 协程消费标准输入回车键，支持交互式运维命令（如手动触发配置重载）。该通道是开发期与运维期的手动干预入口，与信号通道互补。

37.5 GC 调度协同

`gc_manager` 以单例运行，在主循环空闲期调度 GC，避免在音频处理关键路径触发停顿。GC 调度与对话流程协同，感知系统负载动态调节。该协同是实时性优化的底层保障。

三十八、系统演进路线

38.1 当前能力

V1.0 版本已具备：双服务异步架构、六大能力 Provider 矩阵（ASR 14/TTS 20/LLM 11/VAD 2/Intent 2/Memory 6）、统一工具处理器、PowerMem 记忆与图谱、输出门控内容安全、三层配置远程热更新、AEC 全双工、声纹多用户、MQTT 网关集成、OTA 与视觉分析。覆盖 AI 语音陪伴硬件的后端全场景需求。

38.2 能力扩展方向

后续版本演进方向：更多 Provider 适配（持续扩展服务商矩阵）、多模态深度集成（语音+视觉+手势）、本地推理优化（端侧模型量化）、记忆图谱推理增强、内容安全模型升级、插件生态丰富（接入更多 MCP 工具与业务插件）。开闭原则的扩展机制保证演进不改核心。

38.3 性能演进方向

性能优化方向：首字延迟进一步压缩（流式优化与预测生成）、并发承载提升（协程与线程池调优）、内存占用优化（GC 策略精进）、弱网体验优化（自适应编码与速率控制）。`performance_tester` 提供量化基准指导演进。

38.4 部署演进方向

部署演进方向：容器编排深化（Kubernetes 生产级部署）、多地域部署（就近接入降延迟）、边缘计算（端云协同）、Serverless 化探索（HTTP 服务无状态化）。演进保证现有架构平滑过渡。

38.5 合演进方向

合规演进方向：数据保护增强（用户数据权利工具化）、内容安全升级（多模态内容审核）、审计完善（全链路审计追溯）、密钥管理强化（密钥轮换与托管）。演进满足监管与产品合规要求。

三十九、接口适配层设计

39.1 HTTP 处理器基类

`core/api/base_handler.py` 定义 HTTP 处理的通用基类，封装鉴权、参数校验、异常捕获、响应构造的公共逻辑。`OtaHandler` 与 `VisionHandler` 继承基类，仅实现业务逻辑。该设计消除 HTTP 处理器的重复代码，保证鉴权与响应格式一致性。

39.2 鉴权适配

HTTP 处理器经 `base_handler` 统一调用 `core/auth` 完成 JWT 校验。鉴权失败返回 401 统一格式。OTA Token 与视觉 JWT 共用 `auth_key` 根密钥但签名载荷不同，`base_handler` 按 `endpoint` 区分校验逻辑。该适配使鉴权集中可维护。

39.3 响应封装

所有 HTTP 响应经 `base_handler` 统一封装为 JSON 格式，含成功标志、数据载荷、错误信息。成功响应 200，参数错误 400，鉴权失败 401，服务异常 500。响应不泄露敏感信息（内部路径、密钥、堆栈）。该封装保证接口风格一致，便于客户端处理。

39.4 异常适配

`base_handler` 捕获处理器抛出的全部异常，转换为错误响应而非 500 崩溃。业务异常记日志后返回 4xx，系统异常记日志后返回 5xx。该适配保证单个请求异常不影响服务整体可用性，是 HTTP 服务健壮性的保障。

四十、错误分类与恢复设计

40.1 错误分类

系统错误分四类：配置错误（启动期，fail-fast 终止）、网络错误（IO，重试或降级）、Provider 错误（推理失败，降级响应）、协议错误（消息非法，拒绝处理）。每类有对应处理策略，不混用。该分类使错误处理有的放矢。

40.2 可恢复错误

网络错误与 Provider 错误为可恢复错误，捕获后降级处理：TTS 失败回退默认提示音，LLM 失败回复固定话术，ASR 失败提示重新输入。降级不中断连接，下一轮对话自动恢复原流程。该策略保证用户体验连续。

40.3 不可恢复错误

配置错误为不可恢复错误，启动期 fail-fast 终止避免带病运行。auth_key 缺失、ffmpeg 未装、Provider 类名非法均属此类。错误信息明确指向缺失项，便于运维快速修复。该策略保证系统只在健康态服务。

40.4 连接级恢复

单连接异常（队列堆积、Provider 崩溃）由该连接的 ConnectionHandler 捕获，clear_queues 与 reset_audio_states 清理后恢复就绪。不影响其他连接。close 在异常不可恢复时保存摘要后关闭，设备重连开启新会话。该设计是资源隔离的恢复保障。

40.5 错误日志与可追溯

所有错误经 loguru 记录完整上下文：client_id、错误类型、堆栈、发生位置。ERROR 级以上可触发告警。错误日志支持按 client_id 追溯单设备全链路问题，是线上运维的定位依据。audit.log 单独记录安全相关错误。

四十一、二次开发说明

41.1 上游许可

本系统基于开源项目 xiaozhi-esp32-server 二次开发，上游采用 MIT 协议（Copyright (c) 2025 xinnan-tech）。MIT 协议允许商业使用、修改、分发、再授权与出售，唯一要求是在所有副本或重

要部分保留版权与许可声明。本系统在发行物中附上游 LICENSE 作为许可证明，符合 MIT 协议要求。

41.2 独创代码

上海爱予慧科技有限责任公司在上游基础上新增了大量独创代码，主要包括：

- 连接状态机的 AEC 回声消除（_apply_aec / _check_aec_cache_expiry）与音频速率控制（audioRateController）
- Provider 工厂的多服务商适配矩阵（ASR 14 种、TTS 20 种、LLM 11 种、Memory 6 种）与懒加载（modules_initialize / _background_initialize）
- 输出门控内容安全机制（_release_gate_result / _feed_checked_output / _finish_checked_output）
- PowerMem 记忆与知识图谱集成（_initialize_memory / _log_dialogue_summary / 用户画像提取）
- 统一工具处理器（UnifiedToolHandler / UnifiedToolManager）与 MCP 协议对接
- 三层配置与智控台远程热更新（config_loader / update_config）
- GC 管理器（gc_manager）与实时性优化
- 声纹识别（voiceprint_provider）与多用户记忆隔离
- MQTT 音频桥接（_process_mqtt_audio）
- 提示词动态增强（_init_prompt_enhancement / prompt_manager）
- 输出序号管理（output_counter）与句子切分（textUtils / p3）

上述新增代码的著作权归属上海爱予慧科技有限责任公司，原始取得全部权利。上游 MIT 许可证明随本系统发行物一并提供。

41.3 与智控台联动

本系统与智控台（爱予慧AI智能体管理平台）协同：智控台负责设备注册、智能体配置、模型参数管理、NFC 资产管理、用户与反馈管理，本系统在启动时从智控台拉取配置并在运行时接收配置变更。两者构成“管理后台—语音服务”的完整产品形态，共同支撑 AI 语音陪伴硬件的全生命周期运营。

41.4 扩展指南

新增 ASR/TTS/LLM/Memory 服务商：在 core/providers/<能力>/ 下新增实现类继承基类，在 data/.config.yaml 或智控台配置类名即可，无需改业务代码。新增工具能力：在 plugins_func/

functions/ 下新增模块，用 `@register_function` 装饰注册。新增消息类型：在 `textMessageType` 枚举增加成员，在 `textHandler` 下新增处理器。三类扩展均遵循开闭原则，对扩展开放对修改关闭。

41.5 版本与著作权

本系统版本 V1.0，著作权人上海爱予慧科技有限责任公司，单独开发，原始取得，全部权利，未发表。本设计说明书与源代码鉴别材料共同构成本系统软件著作权登记的完整文档材料。

四十二、文档说明

42.1 文档目的

本设计说明书为"爱予慧AI语音交互服务系统 V1.0"软件著作权登记的设计文档，依据《计算机软件著作权登记办法》及中国版权保护中心 R11 登记指南编制。文档全面阐述系统架构、模块设计、数据流转、接口协议与关键技术，与源代码鉴别材料共同构成完整登记材料。

42.2 文档范围

文档覆盖引言、系统概述、架构设计、连接管理、语音流水线、消息处理、Provider 工厂、工具调用、HTTP 接口、记忆图谱、配置系统、关键技术、部署运维、安全设计、数据结构、异常处理、性能设计、接口协议、设备生命周期、可观测性、数据流详设、扩展机制、容量配额、插件系统、会话管理、设备协商、模型管理、版本兼容、MQTT 网关、内容安全、部署架构、灾备高可用、数据隐私、运维自动化、数字人集成、系统初始化、运行时协同、接口适配层、错误分类、演进路线、二次开发、文档说明共四十二章。源代码鉴别材料另册提交。

42.3 上游许可声明

本系统基于开源项目 `xiaozhi-esp32-server` (MIT 协议, Copyright (c) 2025 xinnan-tech) 二次开发。MIT 协议允许商业使用、修改、分发、再授权，唯一要求保留版权与许可声明。本系统发行物附上游 LICENSE 作为许可证明。上海爱予慧科技有限责任公司在上游基础上新增的独创代码著作权归属本公司，原始取得全部权利。

42.4 文档版本

本设计说明书版本 2026-09-01，对应软件版本 V1.0。文档随软件版本演进更新。著作权人上海爱予慧科技有限责任公司。

42.5 联系与归属

本系统著作权人上海爱予慧科技有限责任公司，单独开发，原始取得全部权利，未发表。软件全称"爱予慧AI语音交互服务系统 V1.0"，版本 V1.0。本系统与配套的智控台（爱予慧AI智能体管理平台）、蛋宝宝小程序（爱予慧蛋宝宝AI宠物小程序软件）、数字人（爱予慧数字人交互演示软件）共同构成上海爱予慧科技有限责任公司面向 AI 语音陪伴硬件的完整产品矩阵。各软件著作权独立登记，著作权人统一为上海爱予慧科技有限责任公司。

42.6 文档结语

本设计说明书至此完结。文档基于真实源码与架构编制，内容涵盖系统从接入层到存储层、从启动到运行、从配置到安全的全维度设计。系统基于开源 MIT 项目二次开发，新增独创代码覆盖连接管理、语音流水线、Provider 工厂、工具调用、记忆图谱、内容安全、远程配置等核心模块，形成面向 ESP32 智能硬件的实时语音 AI 交互服务能力。上海爱予慧科技有限责任公司保留本系统全部著作权，原始取得。

42.7 附记

本系统源代码鉴别材料取自 xiaozhi-server 自研业务代码，按语音交互数据流叙事顺序排列：服务入口 app.py → 配置加载 config → 数据模型 models → 连接管理 core/connection、core/auth → 消息处理 core/handle → Provider 流水线 core/providers/{vad,asr,intent,llm,tts,memory,tools,content_safety,vllm} → HTTP/WebSocket 服务 core/http_server、core/websocket_server → REST API core/api → 工具函数 core/utils → 插件函数 plugins_func。该顺序完整呈现"设备接入→音频流转→AI 推理→音频回传"的全链路代码路径，便于审查理解系统业务逻辑。源码每页 60 行（≥50 行下限），共 60 页（前 30 页 + 后 30 页），与本设计说明书共同构成完整的著作权登记鉴别材料。著作权人上海爱予慧科技有限责任公司，单独开发，原始取得全部权利，未发表，版本 V1.0。

四十三、系统测试与验证设计

43.1 测试策略

本系统测试分四层：单元测试（工具函数与 Provider 接口）、集成测试（流水线衔接与消息分发）、端到端测试（完整语音对话）、性能基准（延迟与吞吐）。系统无 pytest 套件，性能基准由 performance_tester 承载，端到端验证由数字人与浏览器测试页支撑。该策略以性能与端到端为核心，契合实时语音系统质量诉求。

43.2 单元测试

核心工具函数（textUtils 句子切分、opus_encoder_utils 编解码、content_safety 校验、wakeup_word 匹配）具备独立可测性，可通过脚本验证边界条件。Provider 工厂 create_instance 的类名解析与实例化逻辑可单元验证。dialogue 的上下文截断与 few-shot 注入可脚本验证。单元测试聚焦纯逻辑无 IO 模块。

43.3 集成测试

集成测试验证模块衔接：receiveAudioHandle 到 VAD 到 ASR 的音频流转、chat 到 LLM 到 TTS 的对话衔接、UnifiedToolHandler 到各工具源的分发、config_loader 三层合并。集成测试通过数字人作为模拟设备，经 WebSocket 发起真实消息，验证各模块协作正确性。该层是发现接口契约问题的核心。

43.4 端到端测试

端到端测试由 digital-human 项目与 test 浏览器页面承载。数字人模拟 ESP32 终端，经 WebSocket 完成完整语音对话：上传音频、识别、推理、合成、回传播放。该测试验证全链路在真实协议下的正确性，是发布前验收的主要手段。test 目录的 test_page.html 提供手动交互测试入口。

43.5 性能基准

performance_tester 目录提供 ASR、LLM、TTS 独立基准测试，量化首字延迟、识别准确率、合成速度、生成吞吐。基准在接入新 Provider 或配置变更后运行，对比指标变化，防止性能回归。performance_tester.py 是统一入口，支持按配置批量测试多 Provider。该机制使性能可量化、可对比、可回归。

43.6 异常场景测试

异常场景验证系统健壮性：Provider 推理失败（TTS 超时、LLM 异常）、连接异常断开、队列堆积、配置缺失。各类异常由对应超时与捕获机制处理，验证不崩溃、可恢复。AEC 缓存过期、GC 停顿等边界场景亦在测试范围。该层保证生产环境的稳定性。

43.7 兼容性测试

多 Provider 矩阵需兼容性测试：各 ASR、TTS、LLM 实现均需验证接口契约一致、流式行为正确。多设备固件版本需验证消息协议前向兼容。Python 3.10 与 3.12 跨版本验证。兼容性测试保证能力矩阵的真实可用性，是系统多服务商适配承诺的质量基础。

四十四、存储与数据库设计

44.1 存储分层

系统存储分三层：OceanBase（MySQL 兼容）承载记忆向量与知识图谱，Redis 承载热配置与连接态缓存，本地文件系统承载临时音频与模型文件。三层各司其职，OceanBase 保证持久化，Redis 保证低延迟，文件系统承载大对象。存储分层平衡了持久性、性能与成本。

44.2 OceanBase 向量存储

记忆向量存储于 OceanBase 的向量列，由 pyobvector 客户端访问。每条记忆记录含 id、content、embedding、metadata 四字段。embedding 由 LLM 生成，维度依模型而定。召回时按余弦相似度检索 top-k，注入对话上下文。oceanbase_vector.py 封装向量读写。

44.3 OceanBase 图谱存储

知识图谱存储为 (subject, predicate, object) 三元组，节点为实体，边为关系。图谱支持关系查询与推理，与向量库协同——向量负责语义相似召回，图谱负责关系推理。oceanbase_graph.py 封装图谱读写。用户画像作为结构化实体存入图谱。

44.4 Redis 缓存

Redis 缓存热配置、连接态、唤醒词响应等高频访问数据。enable_wakeup_words_response_cache 开启唤醒词响应缓存，降低重复唤醒延迟。Redis 减轻 OceanBase 读压力，提升热路径响应速度。缓存设 TTL 保证最终一致。

44.5 本地文件存储

models/ 目录承载本地推理模型文件（如 SenseVoiceSmall）。临时音频文件由 delete_audio 控制是否留存。配置文件 config.yaml 与 data/.config.yaml 在文件系统。本地存储不入版本控制（模型大、配置含密钥）。

44.6 数据生命周期

对话摘要与记忆长期持久化（用户数据权利支持删除）。临时音频处理后即弃。AEC 缓存定期清理。日志按策略滚动。close_connection_no_voice_time 自动关闭空闲连接释放资源。数据生命周期管理保证存储不无限膨胀。

四十五、系统依赖与外部服务设计

45.1 依赖清单

系统依赖分四类：运行时依赖（websockets、aiohttp、aioconsole、PyJWT）、ML 推理依赖（torch、funasr、sherpa_onnx、silero_vad）、存储依赖（pyobvector、Redis 客户端）、工具依赖（loguru、PyYAML）。依赖固定于 requirements.txt，重型 ML 依赖单独标注。Python 3.12 推荐。

45.2 外部服务依赖

系统运行依赖外部服务：OceanBase（记忆存储）、Redis（缓存）、智控台 manager-api（远程配置与设备注册）、各 AI 服务商 API（ASR/TTS/LLM）、ffmpeg（音频编解码）。任一外部服务不可用时系统降级而非崩溃——AI 服务商故障可切换，智控台不可用回退本地配置，OceanBase 不可用降级无记忆模式。

45.3 智控台对接

智控台 manager-api 是本系统的核心外部依赖，提供远程配置下发与设备注册校验。manager-api.url 在 data/.config.yaml 配置，启动时拉取配置覆盖本地，运行时 update_config 支持热更新。设备认证经智控台校验 client_secret。智控台不可用时回退本地配置与离线认证。

45.4 AI 服务商对接

ASR/TTS/LLM 服务商通过 Provider 接口对接，各服务商 API Key 在 data/.config.yaml 或智控台配置。服务商故障由 Provider 异常捕获降级处理，可经智控台切换 selected_module 类名快速换服务商。多服务商矩阵提供充分冗余。

45.5 ffmpeg 依赖

ffmpeg 是音频 Opus 编解码的外部进程依赖，check_ffmpeg_installed 启动时强制校验，缺失则终止启动。编解码通过子进程调用 ffmpeg，避免阻塞事件循环。ffmpeg 是系统运行的硬依赖。

四十六、编码与设计规范

46.1 代码组织

系统代码按职责分层组织：core/ 承载核心逻辑（连接、服务、处理器、Provider、工具），config/ 承载配置与日志，plugins_func/ 承载函数插件，models/ 承载本地模型。每层高内聚低耦合，层间经抽象接口协作。代码组织遵循多小文件原则，单文件聚焦单一职责。

46.2 命名规范

Python 模块与文件用 snake_case（receiveAudioHandle 等历史命名除外），类用 PascalCase（ConnectionHandler、UnifiedToolHandler），函数与方法用 snake_case，常量用 UPPER_SNAKE_CASE。Provider 类名与服务商关联（AliyunASR、TencentTTS）。配置键用 UPPER（ASR、TTS、LLM）。

46.3 异步规范

系统全面异步，IO 与编排均为 asyncio 协程。IO 密集操作（网络、文件）用 async/await，CPU 密集操作（模型推理）经线程池协作避免阻塞事件循环。asyncio Queue 衔接流水线各阶段。每连接独立事件流，互不阻塞。

46.4 错误处理规范

外部交互均做异常捕获与降级，遵循永不因单点异常导致连接崩溃原则。异常在最近可处理层捕获，转为用户可理解响应或日志。TTSException 等自定义异常区分错误类型。fail-fast 用于启动期配置错误，运行期错误降级不中断。

46.5 配置规范

配置全部经 config_loader 三层合并读取，不硬编码。密钥存 data/.config.yaml（gitignore）或智控台远程。配置键在 config.yaml 有注释说明默认值。新增配置项随版本演进，旧配置由默认值补齐保证兼容。

46.6 文档与注释规范

关键模块（ConnectionHandler、Provider 基类、UnifiedToolHandler）有 docstring 说明职责。复杂业务规则（AEC 对齐、输出门控、句子切分）有行内注释说明原因。本设计说明书是系统级设计文档，与代码注释互补，共同保证可维护性。